



UNIVERSITÀ DEGLI STUDI DI MESSINA

DIPARTIMENTO DI SCIENZE MATEMATICHE E INFORMATICHE,
SCIENZE FISICHE E SCIENZE DELLA TERRA

Laurea Triennale in Informatica L31
Curriculum: Tecnologie Informatiche
Tirocinio

TITOLO TESI

Tutor dell'ente ospitante:

Prof. Antonio Celesti

Candidato:

Sandi Russo

Tutor accademico:

Prof. Lorenzo Carnevale

Anno Accademico:

2025/2026

Indice

1	Introduzione	2
1.1	Contestualizzazione	2
1.2	Definizione del problema	3
1.3	Obiettivi del lavoro svolto	4
2	Stato dell'arte	5
2.1	Il modello Property Graph	5
2.2	Neo4j Enterprise e GDS	7
2.3	Graph Embeddings e Graph Neural Networks	8
2.4	GraphSAGE	10
2.4.1	Meccanismo di funzionamento e Scalabilità	11
2.4.2	Pipeline per la link prediction	13
2.4.3	Vantaggi e limitazioni	15
2.5	TransE	15
2.5.1	Meccanismo di funzionamento	15
2.5.2	Pipeline per la link prediction	17
2.5.3	Vantaggi e limitazioni	18
2.6	HashGNN	18
2.6.1	Meccanismo di funzionamento	19
2.6.2	Pipeline per la link prediction	19
2.6.3	Confronto con GraphSAGE	20

2.6.4	Vantaggi e limitazioni	20
3	Materiali e metodi	22
3.1	Il dataset: PharMeBiNet V3	22
3.1.1	Analisi esplorativa del dataset	22
3.2	Infrastruttura del cluster Neo4j	24
3.2.1	Architettura generale	24
3.2.2	Configurazione della memoria	25
3.2.3	Configurazione di rete e WSL2	25
3.2.4	Automazione con Ansible	26
3.3	Caricamento del dataset	26
3.4	Pipeline di estrazione dati	27
3.5	Implementazione dei modelli	28
3.5.1	GraphSAGE	28
3.5.2	TransE	29
3.5.3	HashGNN	30
3.5.4	Training loop	32
3.6	Protocollo di valutazione	33
3.6.1	Metrica principale: AUC-ROC	33
3.6.2	Benchmark di scalabilità	34
3.6.3	Pipeline automatizzata	35
4	Risultati e Discussione	36
4.1	Benchmark infrastrutturale	36
4.1.1	Tempi di estrazione al variare dei nodi attivi	36
4.1.2	Analisi dei risultati infrastrutturali	37
4.2	Risultati della Link Prediction	38
4.2.1	Metriche per modello	38
4.2.2	Confronto grafico	39
4.3	Discussione	40

4.3.1	GraphSAGE: precisione massima, convergenza rapida	40
4.3.2	HashGNN: robusto ma lento	41
4.3.3	TransE: il vincitore dell'efficienza	42
4.3.4	Il trade-off accuratezza-efficienza	42
4.3.5	Confronto con l'ipotesi iniziale	43
5	Conclusioni	45
5.1	Sintesi del lavoro	45
5.2	Discussione dei Risultati	46
5.2.1	Analisi Comparativa delle Architetture Predittive	46
5.2.2	Scalabilità e Overhead Infrastrutturale	46
5.3	Contributi del lavoro	47
5.4	Limitazioni e Sviluppi Futuri	47

Elenco delle figure

2.1	Esempio di Property Graph con nodi etichettati e proprietà	6
2.2	Meccanismo di funzionamento della libreria GDS in Neo4j.	8
2.3	Architettura end-to-end: dal Knowledge Graph eterogeneo (nodi Chemical e Disease) all'estrazione degli embedding nello spazio latente, fino al task di Link Prediction tramite rete neurale (MLP).	8
2.4	Il meccanismo di aggregazione di GraphSAGE: i vicini campionati vengono aggregati e combinati con la rappresentazione del nodo target.	13
2.5	Pipeline architetturale di GraphSAGE: estrazione vettoriale ed inferenza della probabilità di link.	14
2.6	L'intuizione geometrica di TransE: la relazione $\mathbf{r}$ è un vettore di traslazione da $\mathbf{h}$ a $\mathbf{t}$ nello spazio degli embedding.	16
2.7	Pipeline di TransE per la link prediction	17
2.8	Architettura di HashGNN: Chemical e Disease mantengono spazi di embedding separati attraverso tutto l'encoder, convergendo solo nel decoder.	20
3.1	Topologia del cluster Neo4j Enterprise: 1 primary e 3 secondary distribuiti su macchine virtuali Azure. Il client si connette tramite <code>bolt://</code> per la connessione diretta o <code>neo4j://</code> per il routing distribuito.	25
3.2	Esempio di curva ROC: la linea tratteggiata rappresenta un classificatore casuale ($AUC = 0.5$), la curva blu un modello con $AUC \approx 0.99$	34
4.1	Tempi di estrazione nelle tre configurazioni del cluster	37

4.2	AUC-ROC medio con deviazione standard per i tre modelli su 5 run indipendenti	39
4.3	Tempo medio di training per i tre modelli su 5 run indipendenti	40
4.4	Trade-off tra accuratezza (AUC-ROC) ed efficienza computazionale	42

Elenco delle tabelle

2.1	Confronto architetturale tra GraphSAGE e HashGNN.	21
3.1	Grado medio e massimo dei principali tipi di nodo in PharMeBiNet V3. .	24
3.2	Configurazione del cluster Neo4j Enterprise.	24
3.3	Iperparametri condivisi tra i tre modelli.	33
4.1	Tempi di estrazione dati da Neo4j verso PyTorch Geometric al variare della configurazione del cluster.	36
4.2	Risultati della link prediction Chemical→Disease su 5 run indipendenti, 200 epoche.	38
4.3	Risultati dettagliati per singola run.	39

Listings

3.1	Query EDA su PharMeBiNet	23
3.2	Query sul grado dei nodi principali	23
3.3	Configurazione memoria Neo4j (neo4j.conf)	25
3.4	Caricamento del dump PharMeBiNet V3	26
3.5	Creazione del database distribuito con seeding	27
3.6	Query di estrazione del sottografo Chemical→Disease	27
3.7	Costruzione dell'oggetto HeteroData per PyTorch Geometric	28
3.8	Definizione degli embedding learnable per GraphSAGE	28
3.9	Fusione degli spazi di embedding con offset Disease	29
3.10	Inizializzazione degli embedding di TransE	29
3.11	Funzione di scoring di TransE	30
3.12	Margin ranking loss di TransE	30
3.13	Registrazione delle matrici hash fisse in HashGNN	31
3.14	Funzione di hashing LSH in HashGNN	31
3.15	Layer lineari separati per tipo di nodo in HashGNN	31
3.16	Suddivisione training/test con RandomLinkSplit	32
3.17	Esecuzione della pipeline completa	35

Sommario

L'efficacia e la scalabilità di tre algoritmi di Graph Machine Learning vengono analizzate applicandoli a un knowledge graph biomedico di grandi dimensioni. È stato scelto il dataset PharMeBiNet V3 [1], una rete eterogenea con circa 8 milioni di nodi distribuiti su oltre 80 tipi e 53 milioni di relazioni distribuite su 410 tipi distinti, che interconnette composti chimici, geni, proteine e patologie provenienti da oltre 40 banche dati farmacologiche e biologiche.

Il problema studiato è la link prediction tra nodi *Chemical* e *Disease*, biologicamente rilevante nel contesto del drug repurposing, ovvero prevedere relazioni terapeutiche non ancora documentate tra farmaci esistenti e malattie note. Il sottografo estratto per gli esperimenti comprende 594.343 nodi Chemical, 23.580 nodi Disease e 123.361 relazioni dirette tra loro. I tre algoritmi confrontati sono GraphSAGE, TransE e HashGNN, scelti per le loro diverse strategie di rappresentazione, rispettivamente: omogenea con aggregazione dei vicini, traslazionale nello spazio vettoriale, ed eterogenea con proiezioni differenziate per tipo di nodo.

L'infrastruttura sperimentale è un cluster Neo4j Enterprise 2026.04.0 distribuito su quattro macchine virtuali cloud, nella configurazione di 1 primary e 3 secondary. Il benchmark di scalabilità valuta l'impatto del numero di nodi attivi sui tempi di estrazione dei dati verso la pipeline PyTorch Geometric, variando la configurazione tra 1, 2 e 4 nodi.

I risultati mostrano che GraphSAGE raggiunge la precisione più alta con un AUC-ROC medio di 0.993 su cinque run indipendenti, mentre TransE si rivela il modello più efficiente, registrando tempi di training circa tre volte inferiori pur mantenendo una qualità predittiva eccellente ($\text{AUC-ROC} > 0.98$). Sul fronte infrastrutturale, il cluster distribuito non produce speedup significativi su query sequenziali di estrazione batch, evidenziando come il collo di bottiglia sia legato all'overhead di coordinamento di rete piuttosto che alla disponibilità di risorse computazionali.

Acknowledgement

Capitolo 1

Introduzione

1.1 Contestualizzazione

Negli ultimi decenni la quantità di dati prodotti e consumati a livello globale è cresciuta a ritmi senza precedenti. Secondo le stime di Statista, nel 2024 il volume globale delle informazioni create e consumate supererà i 149 zettabyte (10^{21} byte) [2], confermando una crescita esponenziale del volume dei dati prodotti a livello mondiale.

Diversi fattori strutturali spiegano questa esplosione: la diffusione dell’Internet of Things (IoT), la trasformazione digitale della sanità e l’emergere delle tecnologie di *Next-Generation Sequencing* (NGS). Il dato non rappresenta più soltanto un sottoprodotto dell’attività digitale, ma una risorsa strategica che richiede nuovi strumenti di gestione e analisi. Si parla del fenomeno dei **Big Data**, descritto attraverso il concetto delle “cinque V”: il *volume* massiccio dei dataset, la *velocità* con cui i flussi vengono generati e analizzati, la *varietà* delle informazioni, la *veridicità* della fonte e il *valore* dell’analisi [3]. In bioinformatica, questo impatto è ancora più evidente. La ricerca farmacologica e lo studio delle reti metaboliche richiedono l’integrazione di dati molto vari, dove il legame tra le entità biologiche assume un’importanza informatica almeno pari a quella delle entità stesse.

1.2 Definizione del problema

Per decenni, la gestione dei dati è stata dominata dai sistemi **RDBMS** (*Relational Database Management System*), basati su un modello rigorosamente strutturato e su una teoria consolidata che garantisce consistenza e integrità delle informazioni. Tuttavia, mostra limiti evidenti quando viene applicato a domini caratterizzati da forte interconnessione, come le reti biologiche e biomediche.

In tali scenari, le entità non possono essere considerate in modo isolato, ma devono essere interpretate all'interno di una fitta rete di relazioni. Quando una query richiede l'attraversamento di molteplici livelli di connessione, il modello relazionale è costretto a ricorrere a un numero elevato di operazioni di *JOIN*. Questo comportamento introduce un overhead computazionale significativo, che cresce rapidamente con la profondità della query e con la densità della rete, compromettendo la scalabilità delle interrogazioni.

Per far fronte a queste limitazioni, si è progressivamente affermato il paradigma **NoSQL** (*“Not Only SQL”*) [4], termine coniato da Carlo Strozzi nel 1998 per indicare una famiglia eterogenea di sistemi di gestione dei dati caratterizzati da flessibilità dello schema (*schema-agnostic*), scalabilità orizzontale e capacità di distribuzione su infrastrutture commodity. Tale paradigma non sostituisce il modello relazionale, ma lo affianca in contesti in cui la rigidità strutturale rappresenta un vincolo.

All'interno dell'ecosistema NoSQL, i database a grafo hanno assunto un ruolo centrale nella modellazione di sistemi complessi. Essi consentono di rappresentare in modo naturale entità e relazioni, preservando la struttura intrinseca del dominio: i nodi modellano le entità biologiche, mentre gli archi rappresentano le interazioni tra esse. Questa rappresentazione si adatta particolarmente bene alla bioinformatica, dove il valore informativo risiede spesso nelle connessioni piuttosto che nei singoli elementi.

Neo4j rappresenta oggi uno dei database a grafo nativi più diffusi e maturi. Attraverso la libreria *Graph Data Science* (GDS) [5], il sistema mette a disposizione algoritmi scalabili per l'analisi dei grafi e per l'apprendimento automatico su strutture complesse, integrando in un unico ecosistema storage, query engine e pipeline di analisi.

1.3 Obiettivi del lavoro svolto

Il presente lavoro propone un’analisi congiunta teorica e sperimentale basata sul knowledge graph biomedico PharMeBiNet V3 [1], una rete eterogenea di grandi dimensioni composta da circa 8 milioni di nodi e 53 milioni di relazioni, provenienti da molteplici database farmacologici e biologici. La complessità strutturale del grafo lo rende un banco di prova particolarmente rappresentativo per lo studio di algoritmi di Graph Machine Learning in contesti reali.

Il task considerato è la *link prediction* tra nodi di tipo *Chemical* e *Disease*, problematica centrale nel dominio del *drug repurposing*. L’obiettivo consiste nell’identificare potenziali relazioni terapeutiche non ancora documentate, sfruttando la struttura del grafo e le informazioni latenti codificate nelle sue connessioni.

L’analisi si concentra sul confronto tra tre algoritmi: GraphSAGE [6], TransE [7] e HashGNN [8]. Tali modelli sono stati selezionati in quanto rappresentano tre paradigmi distinti di apprendimento su grafi: l’aggregazione di informazioni nei vicini (GraphSAGE), la modellazione geometrica delle relazioni nello spazio latente (TransE) e la gestione esplicita dell’eterogeneità tramite rappresentazioni differenziate per tipo di nodo (HashGNN). Questa diversità metodologica consente di valutare in modo più completo l’impatto delle scelte architetturali sul problema considerato.

Dal punto di vista infrastrutturale, il lavoro prevede la progettazione e l’implementazione di un cluster Neo4j Enterprise distribuito su quattro macchine virtuali in ambiente Azure, configurato secondo un’architettura primary-secondary. L’obiettivo è analizzare l’impatto della scalabilità orizzontale sui tempi di estrazione dei dati necessari alla pipeline di training basata su PyTorch Geometric, variando il numero di nodi attivi nel cluster.

In questo contesto, la valutazione non si limita alla sola accuratezza predittiva dei modelli, ma estende l’analisi alle prestazioni end-to-end della pipeline, con l’obiettivo di quantificare il reale contributo dell’infrastruttura distribuita in scenari di data engineering su grafi biomedici di grande scala.

Capitolo 2

Stato dell'arte

2.1 Il modello Property Graph

Il **Property Graph**, noto anche come *Labeled Property Graph* (LPG), è un modello di dati emerso nei primi anni 2000 per rappresentare reti di entità interconnesse in modo naturale ed efficiente. Un property graph è un multigrafo orientato in cui i vertici rappresentano entità del dominio applicativo e gli archi rappresentano le relazioni tra esse. A differenza del modello relazionale, in cui le associazioni tra entità sono codificate implicitamente tramite chiavi esterne, nel modello a grafo le relazioni esistono come strutture autonome, tipate e arricchibili con proprietà, rendendo la loro navigazione un'operazione nativa e non derivata.

Il modello si compone strutturalmente di due elementi fondamentali:

- I **nodi** (o vertici): rappresentano le entità. Sono caratterizzati da un identificatore univoco, un insieme di *label* che ne classificano la natura semantica e una collezione di proprietà espresse come coppie chiave-valore. Le label consentono di raggruppare i nodi in insiemi omogenei e possono essere modificate dinamicamente senza alterare lo schema del grafo. La possibilità di assegnare zero o più label simultaneamente permette di modellare entità appartenenti a categorie multiple, senza ricorrere a tabelle di associazione aggiuntive.

- Gli **archi** (o relazioni): descrivono le connessioni tra i nodi. Ogni arco possiede un identificatore univoco, un nodo sorgente e un nodo destinazione, un’etichetta che ne definisce il tipo semantico e un’eventuale collezione di proprietà. A differenza dei nodi, ogni arco deve avere un tipo e una direzione esplicita, sebbene quest’ultima possa essere ignorata dal motore di calcolo durante le query che richiedono un attraversamento non orientato.

Questa flessibilità strutturale si adatta perfettamente alla modellazione di domini complessi. Nel contesto biomedico, ad esempio, consente di rappresentare in modo diretto l’interazione tra un composto chimico (nodo con label *Chemical*) e una patologia (nodo con label *Disease*) attraverso un arco tipato (es. *relates_to*), potendo inoltre memorizzare all’interno dell’arco stesso le proprietà specifiche di quella interazione, come il peso statistico o la fonte bibliografica della scoperta.

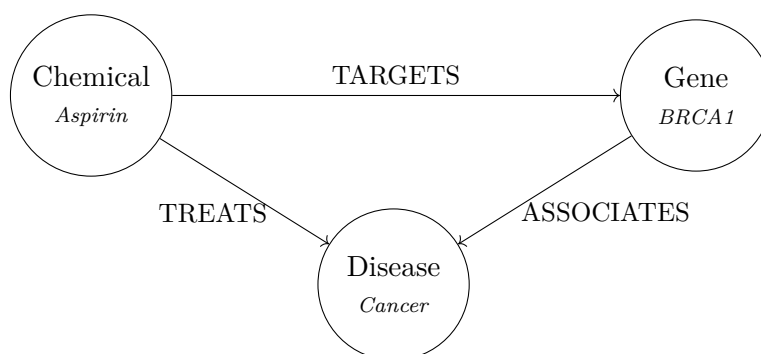


Figura 2.1: Esempio di Property Graph con nodi etichettati e proprietà

Il vantaggio computazionale più rilevante del modello LPG rispetto ai sistemi relazionali emerge nelle query di attraversamento (*graph traversal*). In un RDBMS, la navigazione tra entità correlate richiede operazioni di *JOIN* la cui complessità cresce in modo superlineare con la profondità della query e la cardinalità delle tabelle coinvolte. In un database a grafo nativo, ogni passo di traversata ha un costo costante: dato un nodo, l’accesso ai suoi vicini diretti avviene tramite puntatori fisici senza consultare strutture di indice globali, rendendo il modello LPG particolarmente adatto alla bioinformatica.

2.2 Neo4j Enterprise e GDS

Neo4j è un database a grafo **nativo**, il che significa che implementa il modello a grafo fino al livello di memorizzazione fisica. A differenza dei sistemi che emulano un grafo sopra una tecnologia preesistente, i dati in Neo4j vengono memorizzati esattamente nel modo in cui vengono modellati, senza strati di astrazione intermedi.

Il meccanismo architetturale che rende possibile questa efficienza è l'**index-free adjacency**: ogni nodo memorizza fisicamente i puntatori diretti agli archi incidenti, i quali a loro volta contengono riferimenti immediati ai nodi sorgente e destinazione. Attraversare una relazione non richiede quindi la consultazione di strutture di indice globali, rendendo il costo di ogni singolo passo di traversata $O(1)$ rispetto alla dimensione complessiva del grafo.

Il linguaggio di interrogazione nativo di Neo4j è **Cypher**, un linguaggio dichiarativo ottimizzato per i grafi, concettualmente simile a SQL ma progettato per esprimere pattern strutturali e query di attraversamento in modo intuitivo. Cypher è anche alla base del progetto *openCypher* e ha contribuito alla definizione dello standard **GQL** (*Graph Query Language*).

Per l'analisi algoritmica su larga scala, Neo4j mette a disposizione la libreria **Graph Data Science** (GDS), un'estensione che fornisce implementazioni parallele di algoritmi su grafi esposti come procedure Cypher. La libreria introduce il concetto di **graph projection**: prima dell'esecuzione di un algoritmo, il sottografo di interesse viene proiettato in una struttura in memoria separata, denominata *graph catalog*, ottimizzata per il calcolo analitico. Questo meccanismo disaccoppia il layer transazionale da quello analitico, consentendo l'esecuzione di algoritmi massivamente paralleli senza interferire con le operazioni correnti sul database.

In questa tesi, Neo4j Enterprise 2026.04.0 viene utilizzato come layer di storage distribuito per PharMeBiNet V3. Il cluster è composto da quattro nodi, un primary e tre secondary, e la libreria GDS costituisce la base concettuale del meccanismo di estrazione del sottografo verso PyTorch Geometric, descritto nel capitolo dei metodi.

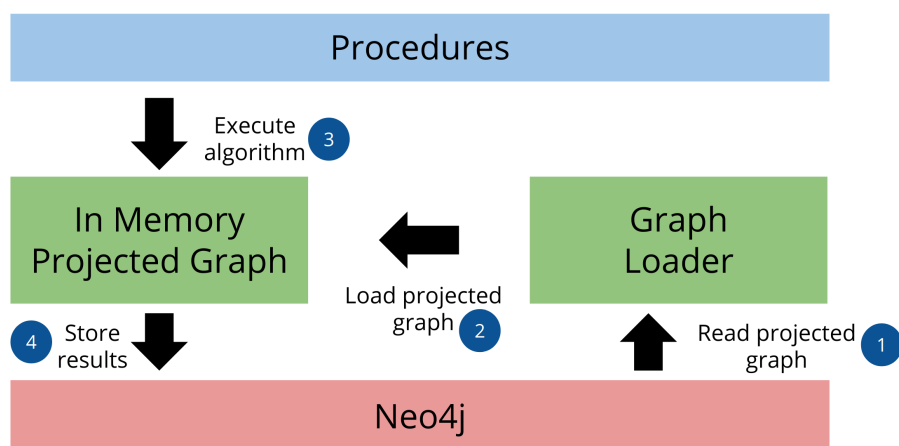


Figura 2.2: Meccanismo di funzionamento della libreria GDS in Neo4j.

2.3 Graph Embeddings e Graph Neural Networks

I **graph embedding** sono funzioni che proiettano i nodi di un grafo in uno spazio vettoriale continuo a bassa dimensionalità, preservando proprietà strutturali e semantiche della rete originale. L'obiettivo è trasformare entità discrete, nodi identificati da un ID o da una stringa, in vettori numerici su cui applicare algoritmi di Machine Learning standard.

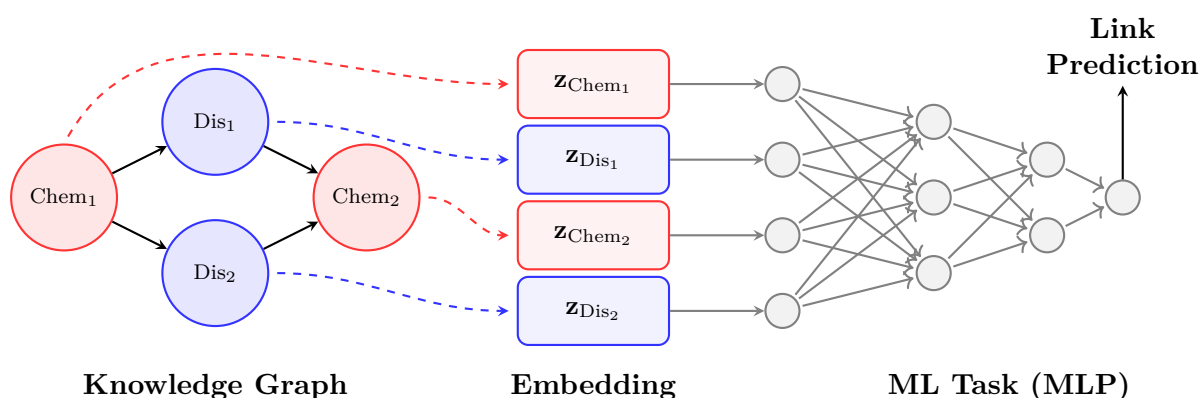


Figura 2.3: Architettura end-to-end: dal Knowledge Graph eterogeneo (nodi Chemical e Disease) all'estrazione degli embedding nello spazio latente, fino al task di Link Prediction tramite rete neurale (MLP).

Le **Graph Neural Networks** (GNN) rappresentano l'evoluzione naturale del deep learning applicato a domini non euclidei. A differenza dei modelli tradizionali, che assumono input strutturati su griglie fisse (come i pixel di un'immagine) o sequenze (come il testo), le GNN sono progettate per operare su topologie irregolari. Esse apprendono gli embedding direttamente dalla struttura del grafo, eliminando la necessità di complesse fasi di *feature engineering* manuale. Il nucleo computazionale che accomuna gran parte di queste architetture è il paradigma del **message passing**: iterativamente, l'embedding di ciascun nodo viene aggiornato aggregando le rappresentazioni vettoriali dei suoi vicini diretti. Ripetendo questo processo per k iterazioni (o layer), l'embedding di un nodo arriva a catturare le dipendenze strutturali e semantiche del suo intero vicinato fino a k salti di distanza nella rete.

Formalmente, dato un nodo v al layer l , la transizione di stato segue lo schema astratto:

$$\mathbf{h}_v^{(l)} = \text{UPDATE}(\mathbf{h}_v^{(l-1)}, \text{AGGREGATE}(\{\mathbf{h}_u^{(l-1)} : u \in \mathcal{N}(v)\})) \quad (2.1)$$

dove $\mathcal{N}(v)$ indica l'insieme dei vicini di v .

Le funzioni AGGREGATE e UPDATE definiscono l'identità dell'architettura: la prima deve essere strettamente invariante rispetto alle permutazioni (poiché l'ordine con cui si leggono i nodi vicini in un grafo non ha rilevanza), mentre la seconda si occupa di combinare l'informazione appena aggregata con lo stato precedente del nodo stesso tramite trasformazioni non lineari.

I tre algoritmi analizzati in questa tesi esplorano lo spazio delle soluzioni predittive secondo filosofie profondamente diverse, offrendo un confronto strutturale completo. Mentre GraphSAGE incarna perfettamente lo schema del *message passing* omogeneo puntando su scalabilità e inferenza induttiva, HashGNN estende tale logica per gestire nativamente la forte eterogeneità di un grafo biomedico. A questi si affianca TransE, che si discosta dal paradigma di aggregazione topologica per operare puramente come modello traslazionale nello spazio latente, fungendo da solida base di confronto (*baseline*) nell'analisi dei Knowledge Graph.

2.4 GraphSAGE

GraphSAGE (*Graph SAmple and aggreGatE*) è un framework per l'apprendimento di graph embedding induttivi, introdotto formalmente da Hamilton et al. nel 2017 [6]. A differenza degli approcci trasduttivi classici (come DeepWalk o Node2Vec), che ottimizzano una matrice di embedding fissa esclusivamente per i nodi osservati durante la fase di training, GraphSAGE compie un cambio di paradigma: invece di memorizzare un vettore per ogni nodo, apprende una **funzione di aggregazione** topologica generalizzabile. Questo approccio permette di generare embedding *on-the-fly* per nodi completamente invisibili durante l'addestramento, rendendo l'algoritmo intrinsecamente adatto a grafi dinamici, reti in continua evoluzione e dataset biomedici di grandi dimensioni.

La motivazione pratica è rilevante in contesti come PharMeBiNet [1], dove nuove evidenze farmacologiche vengono integrate continuamente da decine di banche dati. Un metodo trasduttivo richiederebbe di riaddestrare l'intero modello ad ogni aggiornamento del grafo. GraphSAGE, al contrario, può produrre embedding per un nuovo composto chimico semplicemente osservando il suo vicinato nel grafo aggiornato, senza alcun riaddestramento. Una volta completato il training, il modello addestrato può essere inserito direttamente in una pipeline di inferenza e applicato a nodi non visti, purché questi possiedano un vettore di feature della stessa dimensione e con lo stesso significato semantico usato durante l'addestramento, requisito fondamentale, poiché le operazioni di aggregazione (media, somma) vengono eseguite tra vettori che devono essere comparabili nello stesso spazio.

Un limite importante da tenere presente è che GraphSAGE **non tiene conto delle label dei nodi né dei tipi di relazione**: aggrega i vicini indiscriminatamente, trattando il grafo come se fosse omogeneo. Questo lo rende meno adatto a catturare la ricchezza semantica di grafi fortemente eterogenei come PharMeBiNet, dove 410 tipi di relazione distinti descrivono interazioni biologicamente molto diverse tra loro.

2.4.1 Meccanismo di funzionamento e Scalabilità

L'intuizione alla base di GraphSAGE è che l'identità semantica e strutturale di un nodo possa essere inferita dalla composizione del suo vicinato. In un knowledge graph biomedico, ad esempio, un composto chimico che interagisce con proteine coinvolte in patologie cardiovascolari acquisisce, attraverso il suo vicinato, una rappresentazione che ne riflette il profilo farmacologico. Invece di addestrare un vettore individuale per ciascuna entità, il modello apprende un set di matrici di peso che combinano le feature dei nodi adiacenti, permettendo di generalizzare a nodi mai visti durante il training.

Per superare il noto problema dell'*esplosione del vicinato* (neighborhood explosion), che affligge le tradizionali GNN quando applicate a grafi densi come PharMeBiNet, dove alcuni nodi raggiungono gradi nell'ordine delle decine di migliaia, il *forward pass* di GraphSAGE introduce un meccanismo di campionamento stocastico.

Per ogni layer l , l'elaborazione si articola in due fasi:

1. **Campionamento:** per ogni nodo v , viene selezionato uniformemente un sottoinsieme di dimensione fissa $\mathcal{S}(v) \subseteq \mathcal{N}(v)$ di vicini. La dimensione del campione è un iperparametro che controlla il trade-off tra qualità dell'aggregazione e costo computazionale. Questo vincolo rende il costo per singolo nodo costante e prevedibile, indipendentemente dalla densità locale della rete. In questo modo, anche in presenza di nodi con grado molto elevato, il modello evita l'esplosione del numero di vicini da considerare. Nel caso in cui un nodo sia isolato, ovvero privo di vicini, l'algoritmo recupera informazioni esclusivamente dallo stato corrente del nodo stesso durante lo step di aggregazione.
2. **Aggregazione:** le rappresentazioni dei nodi campionati vengono aggregate in un singolo vettore di contesto, il quale riassume l'informazione proveniente dal vicinato locale. Questo vettore viene poi fuso con la rappresentazione precedente del nodo target tramite concatenazione e trasformazione lineare, così da produrre un nuovo embedding che integra sia le informazioni strutturali sia quelle già codificate al layer precedente.

Formalmente, le equazioni di aggiornamento al layer l sono definite come:

$$\mathbf{h}_{\mathcal{N}(v)}^{(l)} = \text{AGGREGATE}_l (\{\mathbf{h}_u^{(l-1)} : u \in \mathcal{S}(v)\}) \quad (2.2)$$

$$\mathbf{h}_v^{(l)} = \sigma \left(\mathbf{W}^{(l)} \cdot \left[\mathbf{h}_v^{(l-1)} \parallel \mathbf{h}_{\mathcal{N}(v)}^{(l)} \right] \right) \quad (2.3)$$

dove $\parallel$ indica l'operazione di concatenazione vettoriale, $\mathbf{W}^{(l)}$ rappresenta la matrice dei pesi ottimizzabile al layer l , e σ denota una funzione di attivazione non lineare. La scelta della concatenazione rispetto alla semplice somma è cruciale: preserva la distinzione tra lo stato storico del nodo e l'informazione fresca derivante dal vicinato, una distinzione che la somma cancellerebbe collassando i due contributi in un unico vettore indistinguibile. Le funzioni di attivazione supportate includono ReLU, Leaky ReLU e Sigmoid, con ReLU come scelta standard per i layer intermedi.

Con k layer impilati, ogni nodo aggrega informazioni strutturali fino a k salti di distanza nel grafo. Con un solo layer, ogni nodo conosce solo i suoi vicini diretti. Con due layer, la configurazione adottata, ogni nodo integra informazioni sul vicinato del vicinato, catturando pattern strutturali locali di secondo ordine. Aumentare la profondità oltre 2-3 layer introduce il fenomeno dell'**over-smoothing**: le rappresentazioni dei nodi tendono a convergere verso valori simili, perdendo la specificità locale che rende gli embedding utili per task discriminativi. Va inoltre notato che la dimensione degli embedding è direttamente proporzionale alla quantità di informazione catturata: embedding di dimensione maggiore permettono di rappresentare pattern più complessi, ma a costo di un maggiore consumo di memoria.

Affinché l'operazione sia matematicamente robusta, la funzione AGGREGATE deve garantire l'invarianza alle permutazioni. Le implementazioni standard includono l'aggregatore *Mean* (media element-wise dei vettori dei vicini, computazionalmente efficiente), l'aggregatore *Pool* (applicazione di una rete feed-forward a ciascun vicino seguita da max-pooling element-wise, computazionalmente più oneroso) e l'aggregatore *LSTM*, adattato per gestire la natura non ordinata dei vicinati tramite permutazioni casuali.

Nel contesto della *link prediction* su knowledge graph eterogenei, l'applicazione di GraphSAGE richiede la proiezione iniziale di tutti i tipi di nodo in uno spazio vettoriale condiviso. Nel caso specifico di PharMeBiNet, dove i nodi Chemical e Disease non possiedono feature numeriche ma sono identificati solo da stringhe, questa proiezione viene realizzata tramite **embedding learnable**: tabelle di vettori inizializzati casualmente e ottimizzati durante il training. Questa omogeneizzazione forzata, unita all'efficienza computazionale del campionamento, facilita una convergenza rapida, permettendo al modello di sfruttare l'omofilia intrinseca delle reti biomediche già nelle prime epoche di addestramento.

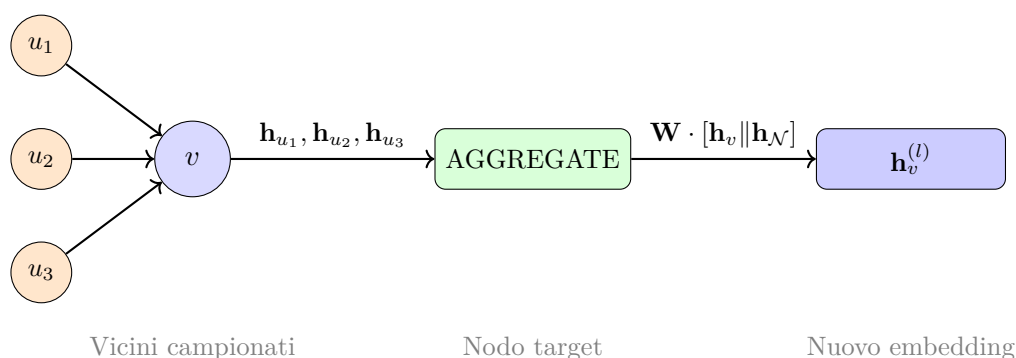


Figura 2.4: Il meccanismo di aggregazione di GraphSAGE: i vicini campionati vengono aggregati e combinati con la rappresentazione del nodo target.

2.4.2 Pipeline per la link prediction

La pipeline si articola in tre componenti distinti.

Encoder: poiché i nodi Chemical e Disease in PharMeBiNet non possiedono feature numeriche, sono identificati solo da stringhe come DB00620 o MONDO:0000004, le feature iniziali sono embedding learnable, una tabella di vettori inizializzati casualmente e ottimizzati durante il training. Gli embedding di Chemical e Disease vengono fusi in un unico tensore omogeneo tramite concatenazione, e gli indici Disease vengono traslati di `num_chemical` posizioni per evitare collisioni. Due layer SAGEConv propagano l'informazione strutturale del vicinato.

Decoder: dati gli embedding $\mathbf{z}_{chem}$ e $\mathbf{z}_{dis}$ di una coppia di nodi, il decoder li concatena e li passa attraverso un MLP (*Multi-Layer Perceptron*) con attivazione finale sigmoid, producendo uno score $p \in [0, 1]$ che rappresenta la probabilità che il link esista.

Training: il modello viene addestrato su link positivi (archi reali) e link negativi generati casualmente, minimizzando la *binary cross-entropy loss*. Il 10% degli archi viene tenuto nascosto per la valutazione finale tramite `RandomLinkSplit`.

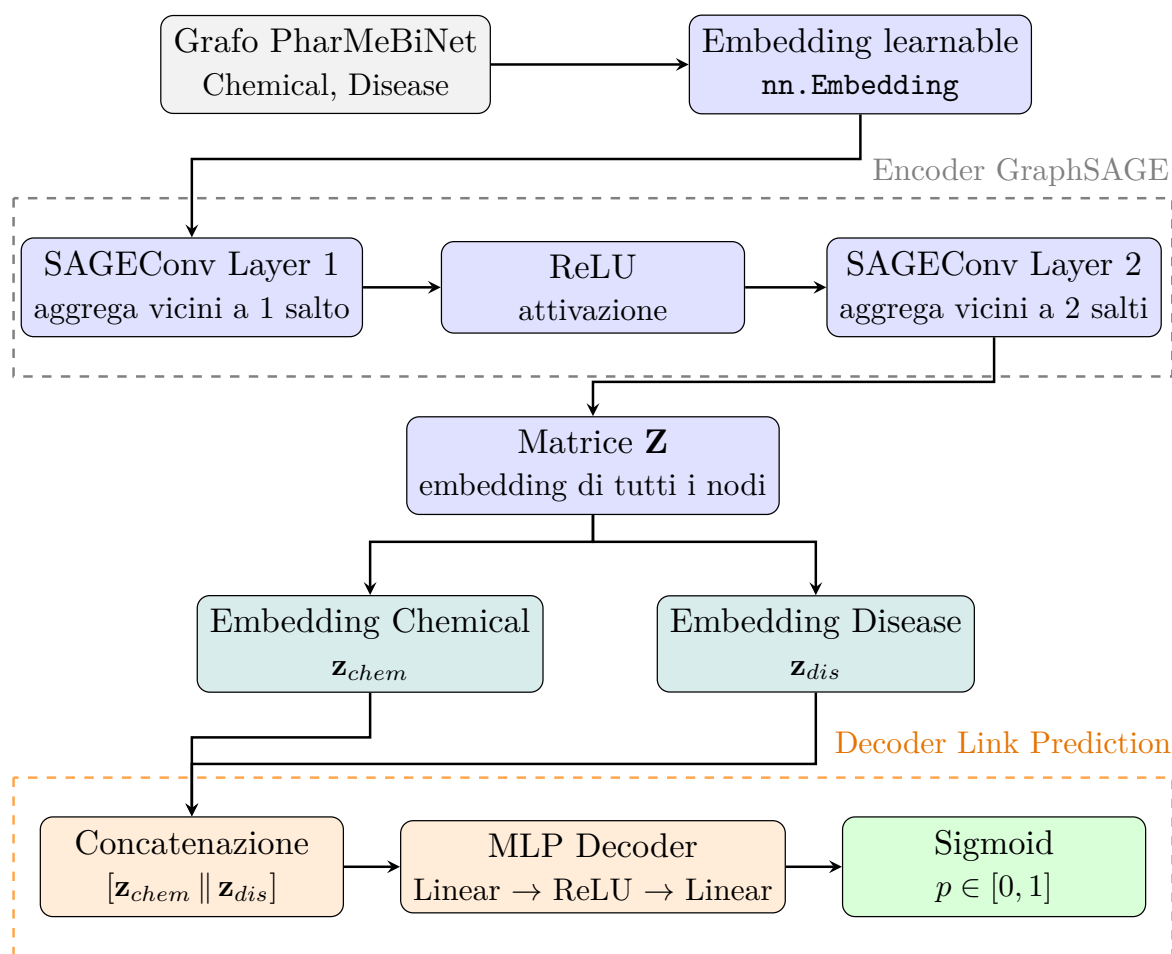


Figura 2.5: Pipeline architetturale di GraphSAGE: estrazione vettoriale ed inferenza della probabilità di link.

2.4.3 Vantaggi e limitazioni

GraphSAGE è induttivo, scalabile e converge rapidamente. Il suo principale limite su PharMeBiNet è che tratta il grafo come omogeneo: non distingue i 410 tipi di relazione presenti nel dataset, aggregando indiscriminatamente vicini di tipo diverso. Su nodi ad alto grado come le proteine (grado medio 232), il campionamento fisso introduce rumore informativo perdendo parte del contesto biologico.

2.5 TransE

TransE (*Translating Embeddings*) è un modello di *knowledge graph embedding* proposto da Bordes et al. nel 2013 [7]. A differenza delle GNN, che aggregano informazione strutturale dai vicini tramite meccanismi di message passing, TransE apprende embedding per entità e relazioni modellando queste ultime come **traslazioni nello spazio vettoriale**. Si tratta quindi di un approccio fondamentalmente diverso da GraphSAGE: non vi è alcuna aggregazione di vicinato. Il grafo viene rappresentato come un insieme di triple (h, r, t) , testa, relazione e coda, e il modello impara a posizionare le entità nello spazio latente in modo che la relazione r corrisponda, idealmente, a un vettore di spostamento da h a t .

Questo paradigma lo rende particolarmente adatto a knowledge graph con molti tipi di relazione semanticamente distinti, come PharMeBiNet, dove 410 tipi di arco descrivono interazioni biologicamente molto diverse, *treats*, *causes*, *disrupts*, *associates*, ciascuna con una semantica propria che GraphSAGE ignora completamente.

2.5.1 Meccanismo di funzionamento

L'intuizione di TransE è geometrica: data una tripla valida (h, r, t) , dove h è il nodo testa, r la relazione e t il nodo coda, il modello ottimizza gli embedding affinché $\mathbf{h} + \mathbf{r} \approx \mathbf{t}$. In altre parole, la relazione è un vettore che, sommato all'embedding della testa, deve avvicinarsi il più possibile all'embedding della coda.

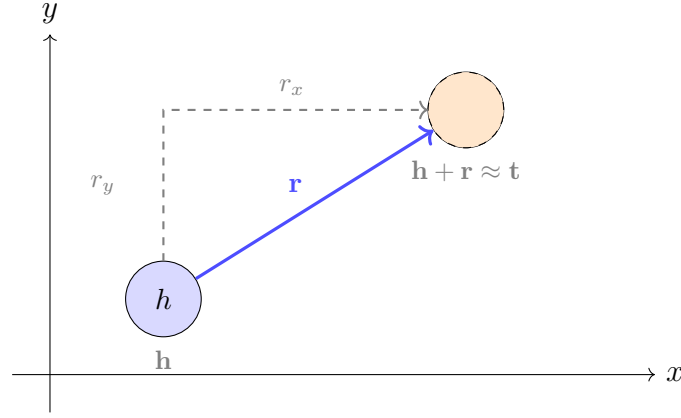


Figura 2.6: L'intuizione geometrica di TransE: la relazione $\mathbf{r}$ è un vettore di traslazione da $\mathbf{h}$ a $\mathbf{t}$ nello spazio degli embedding.

La funzione di scoring per una tripla (h, r, t) è la distanza L2 tra $\mathbf{h} + \mathbf{r}$ e $\mathbf{t}$:

$$f(h, r, t) = \|\mathbf{h} + \mathbf{r} - \mathbf{t}\|_2 \quad (2.4)$$

Un valore basso indica una tripla plausibile, mentre un valore alto indica una tripla improbabile. Il training utilizza una **margin ranking loss**:

$$\mathcal{L} = \sum_{(h,r,t) \in S} \sum_{(h',r,t') \in S'} \max(0, \gamma + f(h, r, t) - f(h', r, t')) \quad (2.5)$$

dove S è l'insieme delle triple positive, S' quello delle triple negative generate per corruzione, e γ è il margine. La loss penalizza i casi in cui la tripla negativa non risulta almeno di γ peggiore della tripla positiva. Durante il training, gli embedding vengono inoltre vincolati o normalizzati per evitare che il modello riduca artificialmente la loss aumentando le norme dei vettori.

Gli embedding vengono normalizzati L2 prima di ogni aggiornamento, per evitare che il modello raggiunga loss bassa semplicemente espandendo le norme dei vettori invece di apprendere una struttura geometrica significativa.

2.5.2 Pipeline per la link prediction

Nel contesto di questa tesi, TransE viene applicato al sottografo Chemical→Disease con un unico tipo di relazione. La pipeline è più semplice di quella di GraphSAGE perché non richiede aggregazione di vicinato: ogni entità e ogni tipo di relazione ha il proprio embedding, e la predizione si basa esclusivamente sulla geometria dello spazio latente.

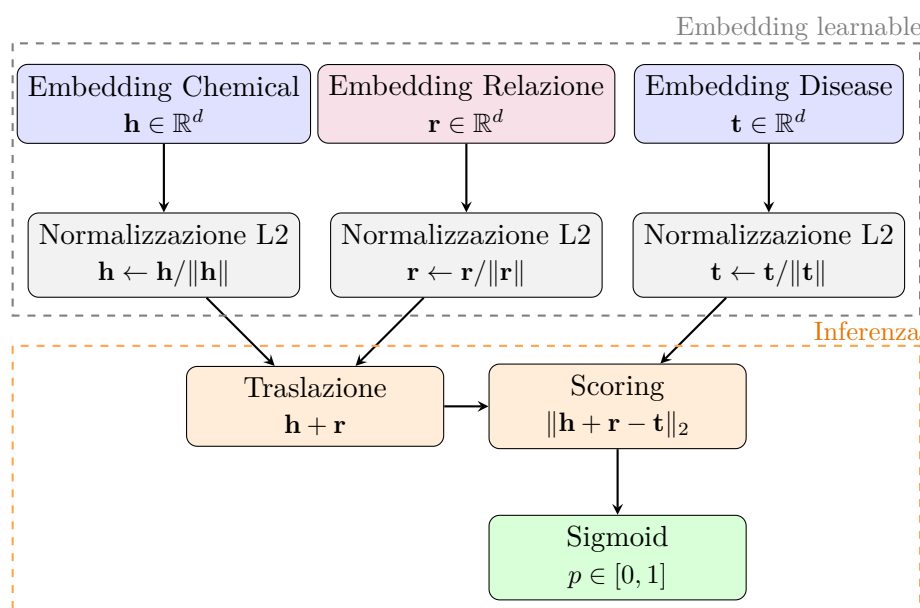


Figura 2.7: Pipeline di TransE per la link prediction

Embedding: ogni nodo Chemical, ogni nodo Disease e ogni tipo di relazione ha un vettore dedicato, inizializzato con distribuzione uniforme in $[-1, 1]$. Non esiste aggregazione di vicinato: ogni entità impara la propria posizione nello spazio latente indipendentemente dalla struttura locale del grafo.

Scoring e inferenza: data una coppia, il punteggio viene calcolato come $-||\mathbf{h} + \mathbf{r} - \mathbf{t}||_2$. La sigmoid trasforma il punteggio in una probabilità tra 0 e 1.

Training: la margin ranking loss confronta triple positive con triple negative generate per corruzione casuale, sostituendo la testa o la coda con un'entità scelta a caso. Il margine $\gamma = 1.0$ forza una separazione minima tra lo score delle triple vere e di quelle false.

2.5.3 Vantaggi e limitazioni

TransE è computazionalmente molto efficiente: non richiede aggregazione di vicinato né propagazione di messaggi, il che lo rende il più veloce dei tre modelli analizzati, con tempi di training circa tre volte inferiori a GraphSAGE. Cattura esplicitamente la semantica delle relazioni tramite il vettore $\mathbf{r}$, il che lo rende teoricamente adatto a knowledge graph con molti tipi di relazione come PharMeBiNet.

Il suo principale limite è che è un modello **trasduttivo**: non generalizza a nodi non visti durante il training. Presenta inoltre difficoltà nel modellare relazioni simmetriche ($h \rightarrow t$ e $t \rightarrow h$ dello stesso tipo) poiché la traslazione è direzionale per definizione, e non cattura informazioni strutturali di ordine superiore come i pattern di vicinato che GraphSAGE sfrutta.

2.6 HashGNN

HashGNN è un algoritmo di graph embedding progettato nativamente per grafi eterogenei [8], e successivamente integrato nella libreria Neo4j GDS [5]. A differenza di GraphSAGE, che fonde i diversi tipi di nodo in un unico spazio omogeneo, HashGNN mantiene **spazi di rappresentazione separati** per ogni tipo di nodo, usando funzioni di hashing casuali per arricchire le feature senza aumentare il numero di parametri apprendibili.

La motivazione è diretta: in un grafo eterogeneo come PharMeBiNet, un nodo Chemical e un nodo Disease rappresentano entità biologicamente incomparabili. Forzarli nello stesso spazio vettoriale fin dall'inizio, come fa GraphSAGE, equivale a proiettare su una mappa comune due lingue diverse senza traduzione. HashGNN evita questo collasso semantico mantenendo coordinate separate per ogni tipo di nodo fino al momento della predizione.

2.6.1 Meccanismo di funzionamento

L’idea centrale di HashGNN è sostituire le proiezioni lineari apprendibili con **proiezioni casuali fisse** ispirate al *Locality-Sensitive Hashing* (LSH) [9]. L’LSH è una famiglia di tecniche che proiettano dati ad alta dimensionalità in spazi a bassa dimensionalità preservando le distanze: punti vicini nello spazio originale rimangono vicini dopo la proiezione con alta probabilità.

Per ogni nodo, l’embedding viene proiettato attraverso K matrici casuali fisse $\mathbf{R}_1, \dots, \mathbf{R}_K$, e i risultati vengono binarizzati tramite la funzione segno:

$$\mathbf{h}_v^{hash} = [\text{sign}(\mathbf{x}_v \mathbf{R}_1^\top) \parallel \dots \parallel \text{sign}(\mathbf{x}_v \mathbf{R}_K^\top)] \quad (2.6)$$

Questo produce K viste binarie diverse dello stesso nodo, ciascuna che cattura aspetti complementari della struttura locale. La binarizzazione tramite sign è computazionalmente economica e produce rappresentazioni sparse, riducendo il costo delle operazioni successive.

Le proiezioni fisse non vengono aggiornate durante il training: solo i layer lineari successivi sono apprendibili, uno per ogni tipo di nodo. Questo disaccoppiamento tra la fase di hashing (fissa, non ottimizzabile) e la fase di proiezione (apprendibile) riduce il numero totale di parametri e limita il rischio di overfitting.

2.6.2 Pipeline per la link prediction

Encoder: gli embedding learnable di Chemical e Disease vengono processati separatamente attraverso le proiezioni hash e i layer lineari dedicati, producendo due matrici distinte $\mathbf{Z}_c$ e $\mathbf{Z}_d$. I due spazi non vengono mai fusi durante l’encoding, questa è la differenza architetturale fondamentale rispetto a GraphSAGE.

Decoder: dati gli embedding $\mathbf{z}_{chem}$ e $\mathbf{z}_{dis}$ di una coppia di nodi, il decoder li concatena e li passa attraverso un MLP con attivazione finale sigmoid. È solo in questo punto che le due rappresentazioni si incontrano, e il MLP deve apprendere la regola di compatibilità tra i due spazi eterogenei.

Training: identico a GraphSAGE, binary cross-entropy su link positivi e negativi generati casualmente, con `RandomLinkSplit` per il test set.

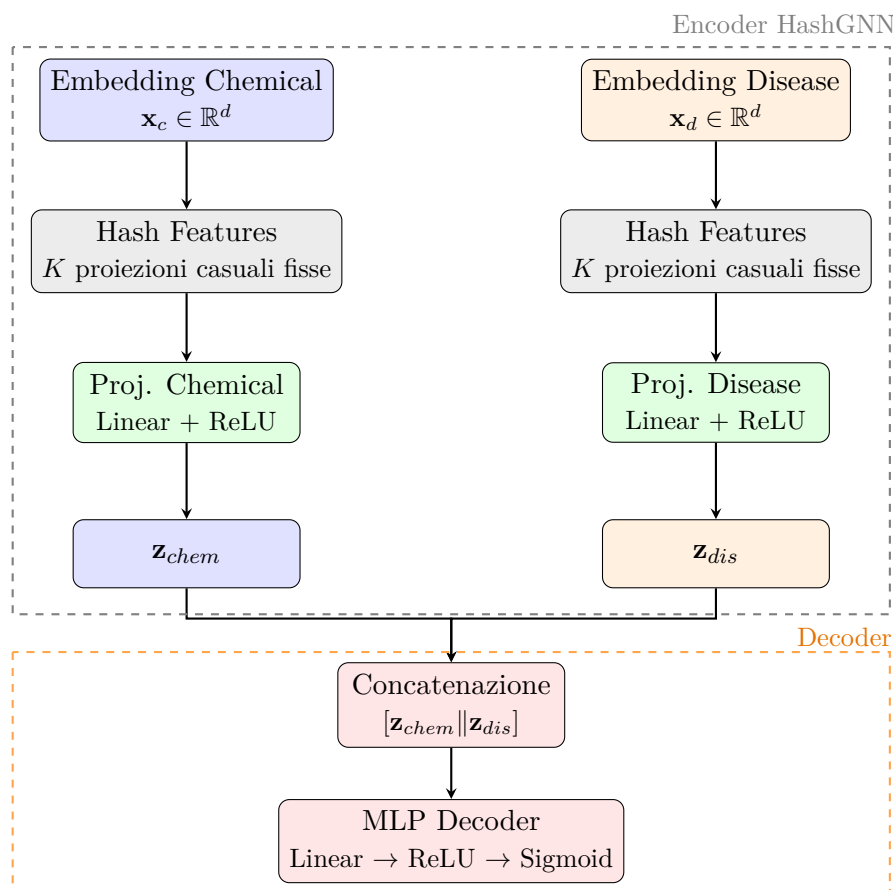


Figura 2.8: Architettura di HashGNN: Chemical e Disease mantengono spazi di embedding separati attraverso tutto l’encoder, convergendo solo nel decoder.

2.6.3 Confronto con GraphSAGE

2.6.4 Vantaggi e limitazioni

HashGNN gestisce nativamente l’eterogeneità del grafo mantenendo proiezioni distinte per ogni tipo di nodo, il che lo rende teoricamente più adatto di GraphSAGE su dataset come PharMeBiNet. Le proiezioni fisse riducono il numero di parametri apprendibili e il rischio di overfitting, e non richiedono backpropagation attraverso la fase di hashing.

Caratteristica	GraphSAGE	HashGNN
Spazio embedding	Unico (omogeneo)	Separato per tipo di nodo
Aggregazione	SAGEConv (apprendibile)	Hash fisso + Linear
Parametri	Più numerosi	Ridotti (hash fisse)
Eterogeneità	Non gestita nativamente	Gestita nativamente
Convergenza	Rapida	Più lenta

Tabella 2.1: Confronto architetturale tra GraphSAGE e HashGNN.

Il principale svantaggio è la **convergenza più lenta**: i layer lineari devono compensare la casualità e la binarizzazione delle proiezioni hash, richiedendo più epoche per stabilizzarsi rispetto a GraphSAGE. Su 200 epoche, questa convergenza più lenta si traduce in un tempo di training circa 1.4 volte superiore e, come vedremo nei risultati, in un'accuratezza finale leggermente inferiore rispetto a GraphSAGE, pur rimanendo statisticamente comparabile.

Capitolo 3

Materiali e metodi

3.1 Il dataset: PharMeBiNet V3

PharMeBiNet (*Pharmacological Medical Biochemical Network*) è un knowledge graph biomedico open-source [1] che integra informazioni provenienti da numerose sorgenti farmacologiche, genomiche e cliniche. Il progetto estende **Hetionet**, un database a grafo biomedico preesistente, aggiungendo dati da sorgenti quali DrugBank per i farmaci, ClinVar per le varianti genomiche, CTD per le interazioni chimico-biologiche, DisGeNET per le associazioni gene-malattia e UniProt per le sequenze proteiche.

È stata utilizzata la versione **V3** del dataset, scaricata da Zenodo [10] e disponibile come dump Neo4j. La versione V3 contiene circa **8 milioni di nodi** distribuiti su **80 label** distinte e **53 milioni di relazioni** di **410 tipi** differenti. Le entità modellate spaziano da farmaci, geni e proteine a malattie, varianti genomiche e reazioni avverse, con relazioni che descrivono interazioni farmaco-farmaco, associazioni gene-malattia e percorsi metabolici [10].

3.1.1 Analisi esplorativa del dataset

Prima di procedere con gli esperimenti è stata condotta un'analisi esplorativa (EDA) del grafo tramite diverse query Cypher eseguita su Neo4j Browser.

```

1 CALL apoc.meta.stats()
2 YIELD labels, relTypesCount, nodeCount, relCount
3 RETURN labels, relTypesCount, nodeCount, relCount

```

Listing 3.1: Query EDA su PharMeBiNet

I risultati confermano la forte eterogeneità del dataset. I nodi dominanti per cardinalità sono le varianti genomiche (**Variant**, **GeneVariant**, **SNV**, circa 3,8 milioni ciascuno), seguiti da modificazioni post-traduzionali (**PTS**, 782 mila), composti chimici (**Chemical**, 594 mila) e interazioni (**Interaction**, 1,1 milioni). I nodi **Disease** ammontano a 23.580 e i **Gene** a circa 193 mila.

L’analisi delle relazioni più dense ha evidenziato che la coppia **Chemical**→**Chemical** RESEMBLES domina il grafo con 31 milioni di archi, circa il 58% del totale, seguita da **Gene**→**GeneVariant** con 6 milioni. Per il task di link prediction, la coppia **Chemical**→**Disease** è stata selezionata per la sua rilevanza biologica nel contesto del drug repurposing e per la disponibilità di dati sufficienti: 123.361 relazioni tra 594.343 nodi Chemical e 23.580 nodi Disease.

Una seconda query ha analizzato il grado medio e massimo dei principali tipi di nodo:

```

1 MATCH (n)
2 WHERE n:Disease OR n:Chemical OR n:Gene OR n:Protein
3 WITH labels(n)[0] AS tipo, count(*) AS totale,
4      avg(size([(n)--() | 1])) AS grado_medio,
5      max(size([(n)--() | 1])) AS grado_max
6 RETURN tipo, totale, round(grado_medio) AS grado_medio, grado_max
7 ORDER BY grado_medio DESC

```

Listing 3.2: Query sul grado dei nodi principali

I gradi massimi sono molto elevati: un singolo nodo Protein arriva a oltre 46.000 connessioni. Questo ha implicazioni dirette sulla scelta degli algoritmi: GraphSAGE, che campiona un numero fisso di vicini, introduce rumore informativo su nodi ad alto grado, mentre HashGNN, che mantiene proiezioni separate per tipo di nodo, è teoricamente più adatto a gestire questa eterogeneità.

Tipo	Totale nodi	Grado medio	Grado massimo
Protein	20.420	232	46.069
Chemical	594.306	112	25.034
Disease	23.580	47	30.419
Gene	193.613	46	36.892

Tabella 3.1: Grado medio e massimo dei principali tipi di nodo in PharMeBiNet V3.

3.2 Infrastruttura del cluster Neo4j

3.2.1 Architettura generale

Il cluster Neo4j Enterprise è stato distribuito su **quattro macchine virtuali Azure**, ciascuna equipaggiata con Windows 10, WSL2 con Ubuntu 22.04 LTS, 8 core e 32 GB di RAM. Neo4j gira all'interno di container Docker su WSL2, con un'unica immagine `docker-compose.yml` distribuita su tutti i nodi tramite Ansible.

IP	Nome	Ruolo
10.0.0.5	neo4j-primary	Primary (lettura e scrittura)
10.0.0.6	neo4j-secondary-1	Secondary (sola lettura)
10.0.0.7	neo4j-secondary-2	Secondary (sola lettura)
10.0.0.8	neo4j-secondary-3	Secondary (sola lettura)

Tabella 3.2: Configurazione del cluster Neo4j Enterprise.

La topologia adottata prevede **1 primary e 3 secondary**. Il nodo primary gestisce tutte le scritture. In caso di guasto, il cluster può eleggere un nuovo primary tra i nodi disponibili tramite il protocollo **Raft**, garantendo la continuità del servizio. I nodi secondary non partecipano al protocollo Raft e vengono usati esclusivamente per scalare le letture distribuite.

La scelta di 4 nodi totali è motivata dall'analisi prestazionale piuttosto che dalla fault tolerance: con 4 nodi il quorum è identico a quello di 3 nodi. L'obiettivo è misurare l'impatto delle secondary sui tempi di lettura distribuita verso la pipeline ML.

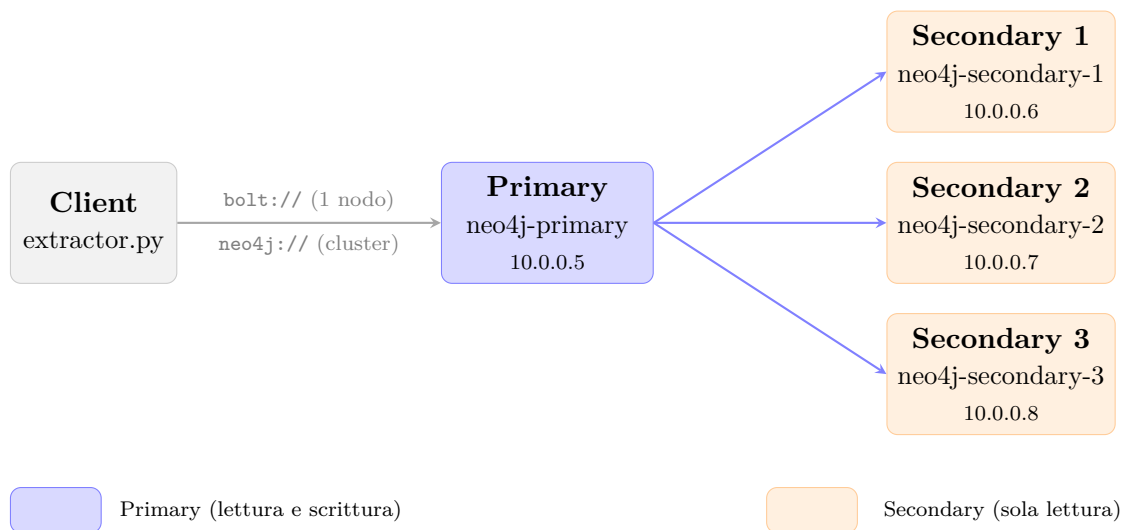


Figura 3.1: Topologia del cluster Neo4j Enterprise: 1 primary e 3 secondary distribuiti su macchine virtuali Azure. Il client si connette tramite `bolt://` per la connessione diretta o `neo4j://` per il routing distribuito.

3.2.2 Configurazione della memoria

Ogni container Neo4j è stato configurato con i seguenti limiti di memoria, bilanciando le esigenze del garbage collector JVM e la velocità di accesso ai dati:

```

1 server.memory.heap.initial_size=8G
2 server.memory.heap.max_size=8G
3 server.memory.pagecache.size=6G

```

Listing 3.3: Configurazione memoria Neo4j (neo4j.conf)

3.2.3 Configurazione di rete e WSL2

La comunicazione tra i nodi del cluster avviene tramite tre porte distinte: la porta 6000 per le transazioni di cluster, la porta 7000 per il protocollo Raft e la porta 7687 per il protocollo Bolt. Poiché ogni nodo gira su WSL2, che ha un indirizzo IP interno diverso dall'IP della macchina Windows, è stato necessario configurare su ogni macchina un port-forwarding tramite `netsh` e uno script PowerShell automatizzato (`ponte.ps1`) che aggiorna dinamicamente i proxy all'avvio, poiché l'IP di WSL2 cambia ad ogni riavvio.

3.2.4 Automazione con Ansible

L'intera gestione del cluster è automatizzata tramite **Ansible**, eseguito dalla primary via SSH verso le secondary. I playbook principali sono:

- `deploy.yml`, distribuisce i file di configurazione e avvia i container su tutti i nodi
- `stop.yml`, ferma i container in modo ordinato
- `reset.yml`, esegue un reset completo rimuovendo tutti i dati
- `setup.yml`, crea la struttura di cartelle necessaria

Il file `.env` su ogni macchina contiene le variabili specifiche del nodo (nome, ruolo, IP) e viene generato da Ansible tramite un template Jinja2.

3.3 Caricamento del dataset

Il dump ufficiale di PharMeBiNet V3 è stato scaricato da Zenodo e copiato nella cartella `data/raw/` sulla primary. Il caricamento è avvenuto tramite un container Docker effimero della stessa immagine Neo4j, montando il volume dati e la cartella del dump:

```
1 docker run --rm -it \  
2   -e NEO4J_ACCEPT_LICENSE_AGREEMENT=yes \  
3   -v $(pwd)/cluster/data:/data \  
4   -v $(pwd)/data/raw:/backups \  
5   neo4j:2026.04.0-enterprise \  
6   neo4j-admin database load pharnebinaet \  
7     --from-path=/backups/ \  
8     --overwrite-destination=true
```

Listing 3.4: Caricamento del dump PharMeBiNet V3

Al termine del caricamento, il database è stato distribuito sulle secondary tramite il meccanismo di seeding nativo di Neo4j Cluster.

```

1 CREATE DATABASE pharmebinet
2   TOPOLOGY 1 PRIMARY 3 SECONDARY
3   OPTIONS {
4     existingData: 'use',
5     existingDataSeedServer: '<SERVER_ID_PRIMARY>'
6   };

```

Listing 3.5: Creazione del database distribuito con seeding

Il parametro `existingDataSeedServer` istruisce il cluster a replicare i dati esistenti dalla primary verso tutte le secondary. Il processo di replica di circa 50 GB ha richiesto alcuni minuti, al termine dei quali tutti e quattro i nodi risultavano **online** per il database `pharmebinet`.

3.4 Pipeline di estrazione dati

L'estrazione del sottografo Chemical→Disease da Neo4j verso PyTorch Geometric [11] è implementata in `src/extractor.py`. La classe `DBExtractor` si connette al cluster tramite il driver ufficiale Neo4j per Python e esegue tre query Cypher sequenziali.

```

1 MATCH (c:Chemical) RETURN DISTINCT c.identifier AS id
2
3 MATCH (d:Disease) RETURN DISTINCT d.identifier AS id
4
5 MATCH (c:Chemical)-[r]->(d:Disease)
6 RETURN c.identifier AS src, d.identifier AS dst

```

Listing 3.6: Query di estrazione del sottografo Chemical→Disease

Per ciascun tipo di nodo viene costruito un dizionario di mapping che traduce gli identificatori stringa (es. `DB00620`) in indici interi contigui a partire da zero. Questo passaggio è essenziale poiché PyTorch Geometric opera esclusivamente su tensori numerici e non su chiavi testuali, richiedendo indici posizionali precisi per la costruzione della matrice `edge_index`.

Il risultato viene incapsulato in un oggetto `HeteroData` di PyTorch Geometric:

```
1 data['Chemical'].num_nodes = len(chem_mapping)
2 data['Disease'].num_nodes = len(dis_mapping)
3 data['Chemical', 'relates_to', 'Disease'].edge_index = \
4     torch.tensor([src_idx, dst_idx], dtype=torch.long)
```

Listing 3.7: Costruzione dell'oggetto `HeteroData` per PyTorch Geometric

L'`edge_index` è un tensore di forma $[2, \text{num_edges}]$ dove la prima riga contiene gli indici sorgente (Chemical) e la seconda gli indici destinatari (Disease). L'adozione di questo formato a coordinate (COO) è lo standard in PyTorch Geometric per rappresentare topologie di rete sparse, garantendo un'occupazione di memoria lineare e operazioni di attraversamento estremamente efficienti.

3.5 Implementazione dei modelli

3.5.1 GraphSAGE

Il modello GraphSAGE è implementato in `src/models/graphsage.py` usando i layer `SAGEConv` di PyTorch Geometric. Poiché i nodi Chemical e Disease non hanno feature numeriche nel grafo, sono identificati solo da stringhe come `DB00620` o `MONDO:0000004`, le feature iniziali sono **embedding learnable**: tabelle di vettori inizializzati casualmente e ottimizzati durante il training tramite backpropagation, una per ogni tipo di nodo. In questo modo, l'informazione semantica originariamente assente viene inferita dinamicamente: man mano che il modello minimizza la funzione di perdita, questi vettori si spostano nello spazio latente fino a codificare l'identità relazionale e le proprietà topologiche di ciascuna entità biomedica.

```
1 self.chem_emb = nn.Embedding(num_chemical, hidden_channels)
2 self.dis_emb  = nn.Embedding(num_disease,  hidden_channels)
```

Listing 3.8: Definizione degli embedding learnable per GraphSAGE

I due spazi di embedding vengono fusi in un unico tensore omogeneo tramite concatenazione prima di essere passati ai layer convoluzionali. Poiché gli indici dei nodi Chemical e Disease partono entrambi da zero, gli indici Disease vengono traslati di `num_chemical` posizioni per evitare collisioni nello spazio globale:

```
1 x = torch.cat([self.chem_emb.weight, self.dis_emb.weight], dim=0)
2 offset = torch.zeros_like(edge_index)
3 offset[1] = num_chemical
4 edge_index_global = edge_index + offset
```

Listing 3.9: Fusione degli spazi di embedding con offset Disease

Due layer `SAGEConv` propagano l'informazione strutturale del vicinato: il primo aggrega i vicini a 1 salto, il secondo a 2 salti. Tra i due layer viene applicata la ReLU. L'encoder produce una matrice con gli embedding aggiornati di tutti i nodi, da cui vengono estratte le righe corrispondenti ai Chemical e alle Disease per la fase di decoding.

Il decoder è un MLP che concatena gli embedding di una coppia (Chemical, Disease) e produce uno score tra 0 e 1 tramite attivazione sigmoid.

3.5.2 TransE

Il modello TransE è implementato in `src/models/transe.py`. A differenza di GraphSAGE, non utilizza nessuna forma di aggregazione di vicinato: mantiene tre tabelle di embedding separate, per i nodi Chemical, per i nodi Disease e per i tipi di relazione, inizializzate con distribuzione uniforme in $[-1, 1]$:

```
1 self.chem_emb = nn.Embedding(num_chemical, embedding_dim)
2 self.dis_emb  = nn.Embedding(num_disease,  embedding_dim)
3 self.rel_emb  = nn.Embedding(num_relations, embedding_dim)
4 for emb in [self.chem_emb, self.dis_emb, self.rel_emb]:
5     nn.init.uniform_(emb.weight, -1.0, 1.0)
```

Listing 3.10: Inizializzazione degli embedding di TransE

Durante il forward pass, tutti gli embedding vengono normalizzati L2 prima del calcolo del punteggio. Questa normalizzazione è essenziale per evitare che il modello raggiunga una loss bassa semplicemente espandendo le norme dei vettori senza apprendere una struttura geometrica significativa.

```
1 def _score(self, src, rel, dst):
2     h = F.normalize(self.chem_emb(src), dim=-1)
3     r = F.normalize(self.rel_emb(rel), dim=-1)
4     t = F.normalize(self.dis_emb(dst), dim=-1)
5     return torch.norm(h + r - t, p=2, dim=-1)
```

Listing 3.11: Funzione di scoring di TransE

Il training usa una **margin ranking loss** con margine $\gamma = 1.0$, che penalizza i casi in cui il punteggio di una tripla negativa non supera quello di una tripla positiva di almeno γ :

```
1 def forward(self, src, rel, dst, src_neg, dst_neg):
2     pos_score = self._score(src, rel, dst)
3     neg_score = self._score(src_neg, rel, dst_neg)
4     return F.relu(self.margin + pos_score - neg_score).mean()
```

Listing 3.12: Margin ranking loss di TransE

In inferenza, il punteggio negativo viene trasformato in probabilità tramite sigmoid: distanza piccola corrisponde a tripla plausibile, quindi lo score viene negato prima di applicare la sigmoid. Poiché PharMeBiNet contiene 410 tipi di relazione ma il sottografo estratto per gli esperimenti include solo relazioni Chemical→Disease, tutte le triple vengono trattate come appartenenti a un unico tipo di relazione (`num_relations = 1`). Questo semplifica il problema ma non permette a TransE di sfruttare appieno la sua capacità di distinguere relazioni semanticamente diverse.

3.5.3 HashGNN

Il modello HashGNN è implementato in `src/models/hashgnn.py`. La differenza architetturale fondamentale rispetto a GraphSAGE è che gli embedding di Chemical e Disease

non vengono mai fusi in un unico spazio omogeneo durante la fase di encoding. Ogni tipo di nodo mantiene il proprio spazio di rappresentazione dall'inizio fino all'ingresso del decoder. Le proiezioni hash sono matrici casuali fisse generate una volta sola e registrate come buffer PyTorch non aggiornabili:

```
1 self.register_buffer('hash_weights',
2     torch.randn(num_hashes, hidden_channels, hidden_channels))
```

Listing 3.13: Registrazione delle matrici hash fisse in HashGNN

L'uso di `register_buffer` garantisce che queste matrici non partecipino all'ottimizzazione, non ricevono gradienti ma vengano correttamente salvate con il modello e spostate su GPU se necessario, esattamente come i parametri apprendibili.

La funzione di hashing applica K proiezioni casuali all'embedding e binarizza il risultato tramite `torch.sign`:

```
1 def hash_features(self, x):
2     return torch.cat([
3         torch.sign(x @ self.hash_weights[i].T)
4         for i in range(self.num_hashes)
5     ], dim=-1)
```

Listing 3.14: Funzione di hashing LSH in HashGNN

Il risultato è un vettore di dimensione $K \times d$ che concatena K viste binarie diverse dello stesso nodo. I layer lineari successivi, separati per Chemical e Disease, trasformano queste rappresentazioni nell'embedding finale:

```
1 self.chem_proj = nn.Linear(hidden_channels * num_hashes, hidden_channels)
2 self.dis_proj  = nn.Linear(hidden_channels * num_hashes, hidden_channels)
3 self.chem_out  = nn.Linear(hidden_channels, out_channels)
4 self.dis_out   = nn.Linear(hidden_channels, out_channels)
```

Listing 3.15: Layer lineari separati per tipo di nodo in HashGNN

Il decoder è strutturalmente identico a quello di GraphSAGE, concatenazione seguita da MLP con sigmoid, ma riceve in input due embedding provenienti da spazi separati anziché da uno spazio fuso.

3.5.4 Training loop

Il training è unificato in `src/train.py` e segue la stessa struttura per tutti e tre i modelli, garantendo un confronto equo. Prima del training, il dataset viene suddiviso tramite `RandomLinkSplit` di PyTorch Geometric [11]:

```
1 transform = T.RandomLinkSplit(  
2     num_val=0.0,  
3     num_test=0.1,  
4     is_undirected=False,  
5     edge_types=[('Chemical', 'relates_to', 'Disease')],  
6     add_negative_train_samples=False,  
7 )  
8 train_data, _, test_data = transform(data)
```

Listing 3.16: Suddivisione training/test con `RandomLinkSplit`

Il 10% degli archi viene tenuto completamente nascosto durante il training e usato esclusivamente per la valutazione finale. I campioni negativi non vengono generati staticamente ma ricreati casualmente ad ogni epoca, il che introduce variabilità e riduce il rischio di overfitting sui negativi.

Per tutti e tre i modelli la procedura di training è:

1. Calcolo delle predizioni su link positivi (archi reali del training set).
2. Generazione di campioni negativi casuali, coppie (Chemical, Disease) non collegate nel grafo, in numero uguale agli archi positivi.
3. Calcolo della loss sui campioni positivi e negativi.
4. Backpropagation e aggiornamento dei pesi tramite Adam optimizer.
5. Dopo 200 epoche, valutazione sul test set con archi mai visti durante il training.

Gli iperparametri condivisi tra i tre modelli sono riportati nella tabella 3.3.

Per garantire la riproducibilità statistica, ogni esperimento è stato ripetuto **5 volte** con seed diverso ($42 + i$ con $i \in \{0, 1, 2, 3, 4\}$), variando sia l'inizializzazione dei pesi

Iperparametro	Valore
Epoche	200
Learning rate	10^{-3}
Optimizer	Adam
Hidden channels	64
Output channels	32
Test split	10%
Run indipendenti	5
Seed iniziale	42

Tabella 3.3: Iperparametri condivisi tra i tre modelli.

che la generazione dei campioni negativi ad ogni run. Questo permette di stimare la varianza dei risultati e distinguere differenze sistematiche tra i modelli da fluttuazioni casuali legate all’inizializzazione.

3.6 Protocollo di valutazione

3.6.1 Metrica principale: AUC-ROC

La metrica principale di valutazione è l’**AUC-ROC** (*Area Under the Receiver Operating Characteristic Curve*), calcolata sugli archi del test set tramite `roc_auc_score` di scikit-learn. La curva ROC rappresenta il trade-off tra il tasso di veri positivi (TPR) e il tasso di falsi positivi (FPR) al variare della soglia di classificazione. L’area sotto questa curva sintetizza la capacità discriminativa del modello in un singolo valore scalare, indipendente dalla soglia scelta.

Un valore di AUC-ROC pari a 0.5 equivale a una classificazione casuale, il modello non distingue link reali da link falsi meglio del caso. Un valore pari a 1.0 indica una separazione perfetta. L’AUC-ROC è la metrica standard per la link prediction su grafi sbilanciati perché non è influenzata dallo sbilanciamento tra classi: in questo esperimento il numero di link negativi generati casualmente è uguale al numero di link positivi reali, ma in uno scenario reale di drug repurposing i link positivi sarebbero una frazione minima del totale.

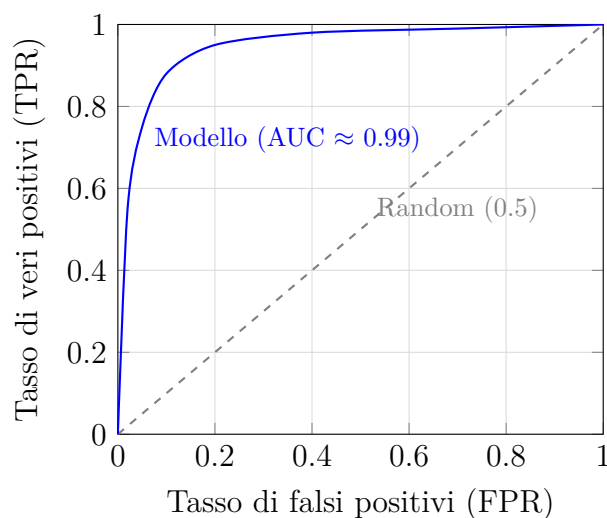


Figura 3.2: Esempio di curva ROC: la linea tratteggiata rappresenta un classificatore casuale ($AUC = 0.5$), la curva blu un modello con $AUC \approx 0.99$

Per ogni modello vengono riportati la media e la deviazione standard dell'AUC-ROC calcolate sulle 5 run indipendenti.

3.6.2 Benchmark di scalabilità

Il benchmark infrastrutturale misura il tempo totale necessario per estrarre l'intero sottografo Chemical→Disease da Neo4j e costruire l'oggetto `HeteroData` per PyTorch Geometric, al variare del numero di nodi attivi del cluster. Le tre configurazioni testate sono:

- **1 nodo:** connessione diretta alla primary tramite protocollo Bolt (`bolt://10.0.0.5:7687`), che bypassa completamente il routing del cluster.
- **2 nodi:** cluster con primary e secondary-1 attivi, connessione tramite protocollo di routing Neo4j (`neo4j://10.0.0.5:7687`).
- **4 nodi:** cluster completo con tutti e quattro i nodi attivi, stessa connessione di routing.

La differenza tra i protocolli `bolt://` e `neo4j://` è sostanziale: il primo stabilisce una connessione diretta a un singolo nodo senza overhead di routing, il secondo attiva il meccanismo di routing table di Neo4j che distribuisce le letture sui nodi disponibili. Questo permette di isolare il contributo del routing distribuito rispetto alla connessione diretta.

Per le configurazioni a 2 nodi, le secondary 2 e 3 (IP 10.0.0.7 e 10.0.0.8) vengono spente tramite Ansible prima della misurazione e riaccese al termine, attendendo 30 secondi per permettere al cluster di riorganizzare la routing table. Il tempo misurato include le tre query Cypher sequenziali e la costruzione del tensore `edge_index`, escludendo il tempo di connessione iniziale e il training dei modelli.

3.6.3 Pipeline automatizzata

L'intera sequenza sperimentale è automatizzata dallo script `run_pipeline.sh`, che esegue in sequenza il benchmark infrastrutturale nelle tre configurazioni e il training dei tre modelli per 5 run indipendenti. I risultati vengono salvati in `results/benchmark.csv` e `results/model_results_final.csv`, e i grafici vengono generati tramite `src/plot_results.py`.

Prima di eseguire la pipeline è necessario caricare le variabili d'ambiente dal file `.env`, in particolare la password del database:

```
1 export $(grep -v '^#' .env | xargs)
2 ./run_pipeline.sh
```

Listing 3.17: Esecuzione della pipeline completa

Questa operazione è necessaria perché le variabili d'ambiente non vengono caricate automaticamente dalla shell all'avvio, e i script Python leggono la password tramite `os.getenv("DB_PASSWORD")` senza default hardcoded nel codice.

Capitolo 4

Risultati e Discussione

4.1 Benchmark infrastrutturale

4.1.1 Tempi di estrazione al variare dei nodi attivi

Il benchmark infrastrutturale misura il tempo necessario per estrarre l'intero sottografo Chemical→Disease da Neo4j e costruire l'oggetto `HeteroData` per PyTorch Geometric [11], al variare del numero di nodi attivi del cluster. Ogni misurazione include le tre query Cypher sequenziali, nodi Chemical, nodi Disease, relazioni, e la costruzione del tensore `edge_index`. I risultati sono ottenuti dall'esecuzione della pipeline completa con il cluster a regime, dopo il caricamento e la distribuzione di PharMeBiNet V3.

Configurazione	Nodi attivi	Protocollo	Tempo medio (s)
1 nodo	Primary only	<code>bolt://</code>	33.8
2 nodi	Primary + Secondary 1	<code>neo4j://</code>	32.1
4 nodi	Cluster completo	<code>neo4j://</code>	33.5

Tabella 4.1: Tempi di estrazione dati da Neo4j verso PyTorch Geometric al variare della configurazione del cluster.

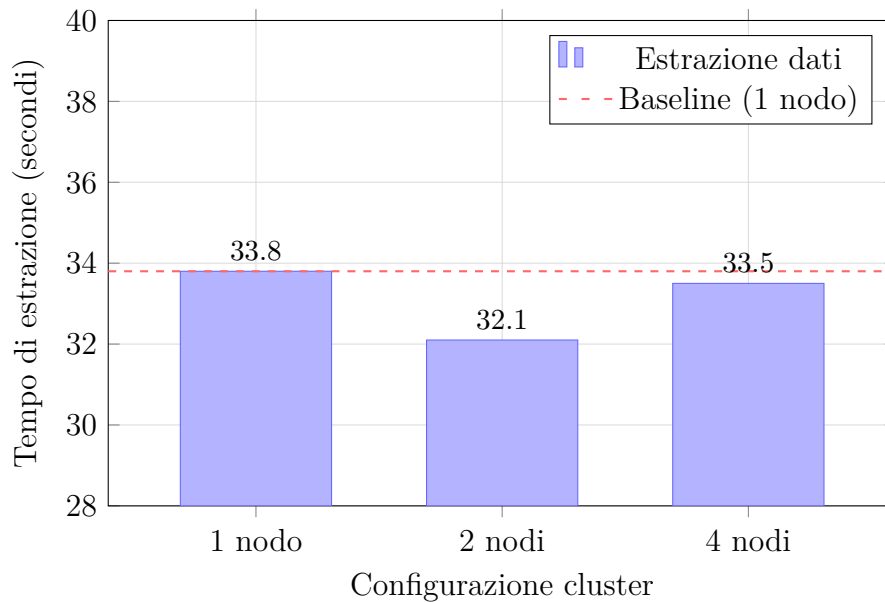


Figura 4.1: Tempi di estrazione nelle tre configurazioni del cluster

4.1.2 Analisi dei risultati infrastrutturali

I tempi risultano sostanzialmente costanti tra le tre configurazioni, con una variazione massima di circa 1.7 secondi. Questo risultato, che potrebbe sembrare deludente, è in realtà coerente con la natura del carico di lavoro e con i principi dei sistemi distribuiti.

Il bottleneck non è la disponibilità di nodi computazionali, ma la **capacità di I/O della singola query sequenziale** su un dataset da circa 50 GB. Le tre query Cypher vengono eseguite una dopo l'altra: non esiste parallelismo a livello applicativo che possa sfruttare la presenza di più nodi. Il router Neo4j distribuisce le letture sulle secondary tramite la routing table, ma con una sola connessione attiva alla volta il vantaggio è marginale e viene parzialmente annullato dall'overhead di coordinazione del cluster.

Un dato interessante emerge dal confronto tra le configurazioni a 2 e 4 nodi: la configurazione a 2 nodi risulta leggermente più veloce della configurazione a 4 nodi. Questo comportamento suggerisce l'esistenza di un *sweet spot* architetturale: con 2 nodi il bilanciamento tra parallelismo di lettura e overhead di routing è ottimale, mentre l'aggiunta di ulteriori secondary introduce latenze di coordinazione che superano il vantaggio.

Va sottolineato che questo risultato non inficia il valore del cluster. Neo4j Enterprise in configurazione distribuita offre vantaggi significativi che questo benchmark non misura direttamente:

- **Fault tolerance:** con la topologia 1 primary e 3 secondary, il servizio rimane operativo anche in caso di guasto di un nodo, senza interruzione delle letture.
- **Disponibilità:** il routing automatico garantisce la continuità delle operazioni anche durante manutenzioni programmate.
- **Scalabilità su carichi paralleli:** con più client che eseguono query simultanee, il cluster distribuisce il carico sulle secondary producendo speedup significativi, uno scenario non testato in questa tesi ma teoricamente più favorevole.

4.2 Risultati della Link Prediction

4.2.1 Metriche per modello

Il training è stato eseguito per **200 epoche** su **5 run indipendenti** con seed diverso ($42 + i$, con $i \in \{0, 1, 2, 3, 4\}$), valutando ciascun modello sul 10% degli archi tenuti nascosti durante il training tramite `RandomLinkSplit`. La tabella 4.2 riporta media e deviazione standard dell'AUC-ROC e del tempo di training per ciascun modello.

Modello	AUC-ROC medio	Std	Tempo medio (s)	Std (s)
GraphSAGE	0.9934	0.0003	448.7	48.2
HashGNN	0.9923	0.0002	624.2	62.1
TransE	0.9839	0.0006	145.2	11.8

Tabella 4.2: Risultati della link prediction Chemical→Disease su 5 run indipendenti, 200 epoche.

La bassa deviazione standard di tutti e tre i modelli, dell'ordine di 10^{-3} o inferiore, indica che i risultati sono stabili e riproducibili tra le run, e non sono artefatti dell'inizializzazione casuale. I risultati per singola run sono riportati nella tabella 4.3.

Run	Modello	AUC-ROC	Tempo (s)
1	GraphSAGE	0.9929	471.8
1	TransE	0.9839	150.6
1	HashGNN	0.9924	629.7
2	GraphSAGE	0.9935	435.2
2	TransE	0.9841	144.3
2	HashGNN	0.9926	612.5
3	GraphSAGE	0.9938	436.5
3	TransE	0.9834	146.5
3	HashGNN	0.9923	623.1
4	GraphSAGE	0.9933	454.3
4	TransE	0.9838	145.8
4	HashGNN	0.9920	625.8
5	GraphSAGE	0.9934	445.5
5	TransE	0.9841	143.0
5	HashGNN	0.9924	630.1

Tabella 4.3: Risultati dettagliati per singola run.

4.2.2 Confronto grafico

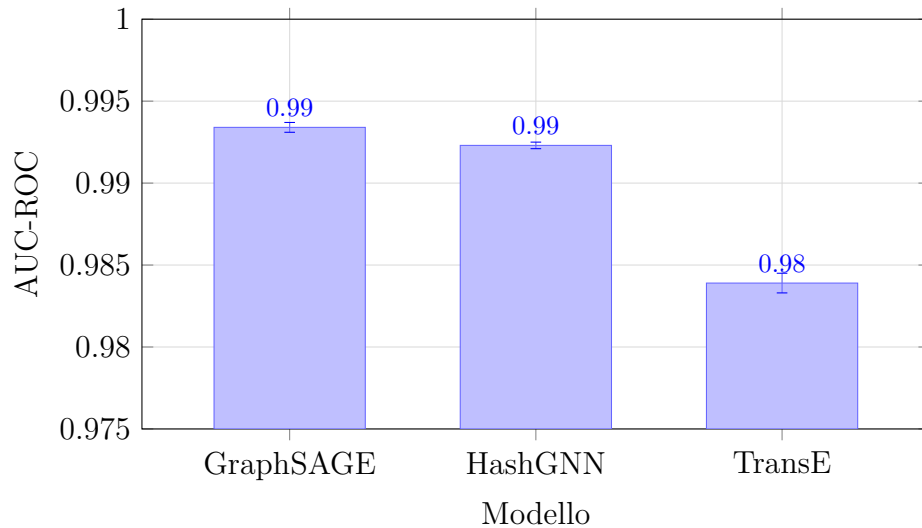


Figura 4.2: AUC-ROC medio con deviazione standard per i tre modelli su 5 run indipendenti

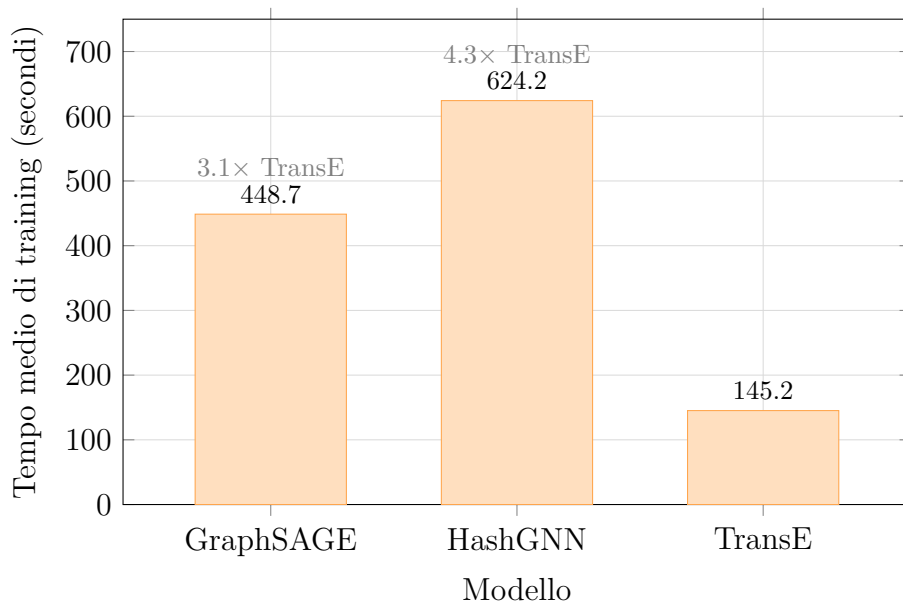


Figura 4.3: Tempo medio di training per i tre modelli su 5 run indipendenti

4.3 Discussione

4.3.1 GraphSAGE: precisione massima, convergenza rapida

GraphSAGE ottiene la precisione più alta con AUC-ROC medio di **0.9934** e una deviazione standard di soli 0.0003, indicando grande stabilità tra le run. Questo risultato è in contrasto con l'ipotesi iniziale, che prevedeva un vantaggio di HashGNN sulla base della sua gestione nativa dell'eterogeneità.

Il vantaggio di GraphSAGE si spiega con due fattori architetturali. Il primo è la **fusione degli spazi di embedding**: unendo Chemical e Disease in un unico spazio latente tramite concatenazione con offset, il modello forza i due tipi di nodo a coesistere nelle stesse coordinate geometriche fin dalla prima epoca. Questo semplifica il compito del decoder MLP, che deve valutare la compatibilità tra coppie di nodi operando in uno spazio già condiviso. Il secondo è la **velocità di convergenza**: le matrici di pesi dei layer SAGEConv sono dense e direttamente ottimizzabili dal gradiente, senza strati intermedi di proiezioni fisse che rallentano la propagazione del segnale.

GraphSAGE mostra però il limite teorico già identificato nell'EDA: non distingue i 410 tipi di relazione presenti nel dataset, aggregando indiscriminatamente vicini di tipo diverso. Su un sottografo limitato a Chemical→Disease questo non costituisce un problema pratico, tutte le relazioni sono dello stesso tipo semantico, ma su task che coinvolgono l'intero grafo eterogeneo il degrado di prestazioni sarebbe atteso e motivato.

4.3.2 HashGNN: robusto ma lento

HashGNN segue GraphSAGE con AUC-ROC medio di **0.9923**, una differenza di soli 0.0011 punti. Dal punto di vista statistico, con una deviazione standard di 0.0002, i due modelli si sovrappongono parzialmente nelle loro distribuzioni di risultati: la differenza è reale ma marginale, e in un'applicazione pratica di drug repurposing sarebbe difficile motivare la preferenza per uno rispetto all'altro sulla base della sola accuratezza.

Il costo di questa robustezza è il **tempo di training**: con una media di 624 secondi per 200 epoche, HashGNN è circa 1.4 volte più lento di GraphSAGE e circa 4.3 volte più lento di TransE. La causa risiede nella natura delle proiezioni hash fisse: i layer lineari successivi devono fare un lavoro più intenso per interpretare le rappresentazioni binarizzate, richiedendo più iterazioni per stabilizzarsi. Non ricevendo gradienti attraverso le matrici hash, che sono buffer fissi, l'unico canale di aggiornamento per l'encoding sono i layer lineari `chem_proj` e `dis_proj`, che partono da una rappresentazione già fortemente trasformata e devono imparare a correggerne la casualità.

L'ipotesi che HashGNN potesse superare GraphSAGE su PharMeBiNet non si è verificata. Questo non è un fallimento sperimentale: in ambito accademico, **un'ipotesi smentita da dati raccolti con metodologia rigorosa ha lo stesso valore scientifico di un'ipotesi confermata**. Il risultato suggerisce che, su questo specifico sottografo Chemical→Disease, l'omofilia biologica, la tendenza dei composti chimici a legarsi a malattie strutturalmente simili, è sufficientemente forte da essere catturata da un modello omogeneo come GraphSAGE, senza necessità di gestire esplicitamente l'eterogeneità.

4.3.3 TransE: il vincitore dell'efficienza

TransE ottiene AUC-ROC medio di **0.9839**, circa un punto percentuale sotto GraphSAGE, con un tempo di training medio di soli **145 secondi**. Questo lo rende il più veloce dei tre modelli di un margine molto significativo: tre volte più rapido di GraphSAGE e oltre quattro volte più rapido di HashGNN.

La velocità di TransE deriva dalla sua architettura essenziale: non esiste aggregazione di vicinato, non esistono layer convoluzionali, non esiste message passing. Ogni aggiornamento coinvolge solo i lookup nelle tabelle di embedding e il calcolo di una distanza L2, operazioni computazionalmente banali anche su dataset di centinaia di migliaia di nodi.

Va notato che su questo sottografo TransE non può esprimere il suo principale vantaggio teorico, la modellazione esplicita di relazioni semanticamente diverse tramite il vettore $\mathbf{r}$, perché tutte le relazioni Chemical→Disease vengono trattate come un unico tipo. Su un sottografo che includesse tutti i 410 tipi di relazione di PharMeBiNet, TransE avrebbe verosimilmente un vantaggio qualitativo maggiore rispetto ai competitor.

4.3.4 Il trade-off accuratezza-efficienza

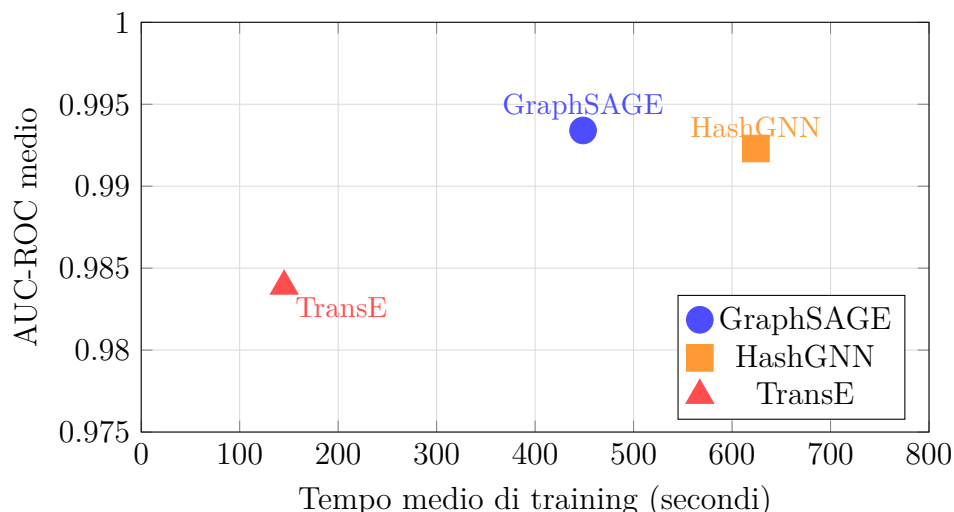


Figura 4.4: Trade-off tra accuratezza (AUC-ROC) ed efficienza computazionale

La figura 4.4 mostra la posizione dei tre modelli nello spazio accuratezza-efficienza e rende immediatamente evidente una considerazione importante: **GraphSAGE domina HashGNN in senso di Pareto**. A parità di quadrante ad alta accuratezza, GraphSAGE ottiene risultati migliori in meno tempo, non esiste quindi nessuna ragione pratica per preferire HashGNN a GraphSAGE sullo specifico task e dataset analizzati.

TransE si posiziona invece in un quadrante separato, offrendo un'alternativa genuinamente diversa: se il vincolo è il tempo di training, come in scenari industriali con aggiornamenti frequenti del grafo o con dataset molto più grandi, sacrificare 0.01 di AUC per ottenere un training 3 volte più veloce può essere la scelta corretta.

4.3.5 Confronto con l'ipotesi iniziale

L'ipotesi formulata prima degli esperimenti prevedeva che HashGNN avrebbe ottenuto le prestazioni migliori su link prediction, seguito da TransE e infine GraphSAGE. I dati mostrano un ordine diverso in termini di accuratezza: GraphSAGE primo, HashGNN secondo a distanza minima, TransE terzo.

Le ragioni di questo scostamento sono identificabili e scientificamente interessanti. La fusione omogenea degli embedding di GraphSAGE si rivela vantaggiosa proprio per la link prediction, dove il decoder deve confrontare direttamente le rappresentazioni di due tipi di nodo diversi: avere già i due tipi nello stesso spazio semplifica questo confronto. HashGNN, mantenendo gli spazi separati fino all'ultimo layer, affida al decoder un compito geometricamente più difficile. Inoltre, 200 epoche si sono dimostrate sufficienti per la convergenza di GraphSAGE ma probabilmente non ottimali per HashGNN, che per sua natura richiede più iterazioni per compensare la casualità delle proiezioni hash.

Sul fronte infrastrutturale, l'ipotesi di uno speedup misurabile con 4 nodi rispetto a 1 nodo non si è verificata nelle condizioni di questo esperimento. Il risultato è coerente con la teoria dei sistemi distribuiti e costituisce un contributo empirico: dimostra che la scalabilità orizzontale di Neo4j non produce vantaggi su pipeline di estrazione batch sequenziali, e che il beneficio reale del cluster riguarda la disponibilità e la fault tolerance piuttosto che il throughput su singole query.

Entrambe le ipotesi smentite, la superiorità di HashGNN e lo speedup del cluster, sono risultati scientificamente validi. Dimostrano che le proprietà teoriche degli algoritmi e dei sistemi distribuiti non si traducono automaticamente in vantaggi pratici quando il contesto sperimentale non soddisfa le condizioni necessarie: un sottografo omogeneo nel caso di HashGNN, un carico parallelo nel caso del cluster Neo4j.

Capitolo 5

Conclusioni

5.1 Sintesi del lavoro

Questo lavoro ha affrontato congiuntamente due problematiche centrali nell'ambito della bioinformatica computazionale e dell'ingegneria dei dati: la valutazione comparativa di architetture di Graph Machine Learning applicate a un *knowledge graph* biomedico di grandi dimensioni e la misurazione empirica dell'impatto della scalabilità orizzontale di un database a grafo sui tempi di alimentazione di una pipeline analitica.

Il dataset scelto, PharMeBiNet V3, caratterizzato da oltre 8 milioni di nodi e 53 milioni di relazioni, ha costituito un banco di prova sfidante per via della sua elevata eterogeneità strutturale e semantica. Il dominio applicativo si è concentrato sulla *link prediction* tra composti chimici (*Chemical*) e patologie (*Disease*), un task predittivo di vitale importanza nel contesto del *drug repurposing* finalizzato alla scoperta di relazioni terapeutiche latenti.

Dal punto di vista dell'ingegneria del software, l'infrastruttura sperimentale si è basata su un cluster Neo4j Enterprise distribuito su quattro macchine virtuali cloud in topologia *primary-secondary*. L'intera architettura è stata orchestrata e automatizzata tramite Ansible e container Docker, garantendo la totale riproducibilità degli esperimenti attraverso una pipeline *end-to-end*.

5.2 Discussione dei Risultati

5.2.1 Analisi Comparativa delle Architetture Predittive

Il confronto tra i modelli ha evidenziato risultati controintuitivi rispetto all'ipotesi di partenza. **GraphSAGE** [6] si è dimostrato l'algoritmo più accurato, registrando un AUC-ROC medio di 0.9934. Il successo di un approccio architetturale omogeneo su un grafo eterogeneo si spiega con la forte omofilia biologica presente nel sottografo estratto: la proiezione forzata delle *feature* in un unico spazio tensoriale condiviso ha semplificato l'ottimizzazione del *decoder* MLP, favorendo una convergenza rapida e precisa.

HashGNN [8], pur essendo progettato nativamente per gestire l'eterogeneità topologica, ha ottenuto un AUC-ROC medio di 0.9923. Tale scarto (0.0011) risulta statisticamente trascurabile, ma si accompagna a tempi di addestramento circa 1.4 volte superiori rispetto a GraphSAGE. Ne consegue che il vantaggio teorico offerto dal partizionamento dei messaggi per tipologia di nodo non si traduce in un reale guadagno prestazionale sul problema e sul perimetro dati analizzato.

Infine, **TransE** [7] ha riportato un'accuratezza leggermente inferiore (AUC-ROC 0.9839), confermandosi tuttavia come il modello nettamente più efficiente della suite. Con tempi di addestramento circa tre volte inferiori rispetto a GraphSAGE, TransE rappresenta il miglior compromesso (*trade-off*) tra qualità predittiva e costo computazionale per l'applicazione su reti in scala industriale.

5.2.2 Scalabilità e Overhead Infrastrutturale

Il *benchmark* infrastrutturale ha dimostrato che la scalabilità orizzontale in fase di estrazione sequenziale *batch* non produce benefici in termini di tempo di esecuzione. Le misurazioni sulle configurazioni a 1, 2 e 4 nodi si sono mantenute costanti, con oscillazioni massime nell'ordine di 1.7 secondi. Questo comportamento certifica che il vero collo di bottiglia (*bottleneck*) dell'operazione risiede nell'I/O di rete e nel coordinamento della transazione da parte del *router* del cluster, non nella potenza di calcolo disponibile.

Tuttavia, il reale valore aggiunto dell’architettura a 4 nodi si manifesta sul piano dell’affidabilità (*fault tolerance*). La ridondanza dei dati sui nodi *secondary* ha garantito la continuità operativa del database.

5.3 Contributi del lavoro

La presente tesi apporta tre contributi tecnico-scientifici distinti:

- **Validazione empirica di algoritmi su dominio reale:** Un’analisi rigorosa (su 5 cicli di validazione indipendente) del comportamento di GraphSAGE, TransE e HashGNN su un task biomedico, offrendo linee guida applicative basate sul *trade-off* tra accuratezza e tempi di calcolo.
- **Ingegnerizzazione infrastrutturale *Enterprise*:** Il rilascio di un’infrastruttura completa e versionata (*Infrastructure as Code* tramite Ansible) per il *deploy*, lo spegnimento chirurgico e il monitoraggio di un cluster distribuito Neo4j dedicato all’alimentazione di flussi di Machine Learning.
- **Profilazione dell’I/O su grafi:** La quantificazione misurabile dell’overhead di rete su database a grafo distribuiti durante carichi di estrazione sequenziali, sfatando il mito della scalabilità lineare *out-of-the-box* per ogni tipologia di interrogazione.

5.4 Limitazioni e Sviluppi Futuri

Nonostante i risultati ottenuti, l’analisi presenta limitazioni che offrono interessanti spunti per espansioni future.

In primo luogo, l’addestramento si è concentrato esclusivamente sulla topologia Chemical→Disease. Un’estensione naturale prevede l’inclusione di ulteriori entità biologiche (es. *Gene*, *Protein*) e tipi di arco, scenario in cui i modelli puramente eterogenei come HashGNN o modelli basati su meta-path (es. HAN) [12] potrebbero esprimere il loro pieno potenziale.

In aggiunta, la sperimentazione di HashGNN su un numero significativamente superiore di iterazioni (300-500 epoche) potrebbe valutare le reali dinamiche di convergenza a lungo termine del modello.

Sul fronte dell'ingegneria dei dati, il *benchmark* limitato alle query sequenziali invita a una profilazione più aggressiva tramite interrogazioni parallele asincrone generate da molteplici *client*, condizione ideale per saturare le risorse di un cluster a 4 nodi e misurarne i reali vantaggi in lettura. Un paradigma architetturale speculare consisterebbe nell'eliminare totalmente la latenza di estrazione demandando l'addestramento alla libreria *Graph Data Science* (GDS) interna a Neo4j, eseguendo le elaborazioni vettoriali direttamente sulle istanze del database.

Infine, la concretizzazione ultima di questa pipeline risiede nel suo utilizzo farmaceutico attivo: analizzare le code della distribuzione delle probabilità per identificare i collegamenti Chemical→Disease altamente probabili ma inesistenti nel database attuale, proponendoli alla comunità medica come candidati primari per validazioni *in vitro* e veri e propri *drug repurposing*.

Bibliografia

- [1] Cassandra Königs, Marcel Friedrichs, and Theresa Dietrich. The heterogeneous pharmacological medical biochemical network PharMeBINet. *Scientific Data*, 9(1):393, 2022.
- [2] Statista. Volume of data/information created, captured, copied, and consumed worldwide from 2010 to 2013, with forecasts from 2024 to 2027. <https://www.statista.com/statistics/871513/worldwide-data-created/>, 2024. Accesso: Giugno 2026.
- [3] Paolo Atzeni, Stefano Ceri, Stefano Paraboschi, and Riccardo Torlone. *Basi di Dati*. McGraw-Hill Education, 5 edition, 2018.
- [4] Adam Fowler. *NoSQL For Dummies*. John Wiley & Sons, 2015.
- [5] Neo4j Inc. Neo4j graph data science library. <https://neo4j.com/docs/graph-data-science/current/>. Accesso: Giugno 2026.
- [6] Will Hamilton, Zhitao Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- [7] Antoine Bordes, Nicolas Usunier, Alberto Garcia-Duran, Jason Weston, and Oksana Yakhnenko. Translating embeddings for modeling multi-relational data. In *Advances in Neural Information Processing Systems*, 2013.

- [8] Neo4j Inc. HashGNN – Neo4j graph data science. <https://neo4j.com/docs/graph-data-science/current/machine-learning/node-embeddings/hashgnn/>. Accesso: Giugno 2026.
- [9] Piotr Indyk and Rajeev Motwani. Approximate nearest neighbors: Towards removing the curse of dimensionality. In *Proceedings of the 30th Annual ACM Symposium on Theory of Computing*, pages 604–613, 1998.
- [10] Cassandra Königs, Marcel Friedrichs, and Theresa Dietrich. PharMeBINet – the heterogeneous pharmacological medical biochemical network. <https://doi.org/10.5281/zenodo.5816976>, 2022. Accesso: Giugno 2026.
- [11] Matthias Fey and Jan E. Lenssen. Fast graph representation learning with PyTorch Geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.
- [12] Xiao Wang, Houye Ji, Chuan Shi, Bai Wang, Yanfang Ye, Peng Cui, and Philip S. Yu. Heterogeneous graph attention network. In *The World Wide Web Conference*, pages 2022–2032, 2019.